

LegoLog: A configurable transparency log

Vivian Fang*, Emma Dauterman^{†‡}, Akshay Ravoort*, Akshit Dewan* and Raluca Ada Popa*

**Department of Electrical Engineering and Computer Science
UC Berkeley, Berkeley, USA*

Emails: {v.fang, akshit, akshayr, raluca.popa}@berkeley.edu

*†Department of Electrical Engineering and Computer Science
MIT, Boston, USA*

*‡Department of Computer Science
Stanford University, Stanford, USA
Email: edauterman@cs.stanford.edu*

Abstract—Transparency logs are critical for a wide range of applications, from web certificates to end-to-end encrypted messaging. Today, many transparency log designs exist for various applications and workloads, and developers must fully understand the design space to find the best design for their needs. Worse, if a developer needs a transparency log for an application and workload without an existing transparency log, the developer (who might not be an expert) must design a new log. To address these challenges, we introduce the paradigm of a *configurable transparency log*, which takes as input a description of the application workload and constraints of different entities and automatically outputs a transparency log uniquely suited to the application. We present the first configurable transparency log design, LegoLog, which we implement and empirically evaluate end-to-end for three specialized transparency logs. We also show that LegoLog can express six different applications, and we compare the asymptotic complexity of LegoLog and existing transparency logs tailored to individual applications. We find that configurability does not come at the cost of performance: LegoLog can capture a variety of applications while performing comparably to existing, special-purpose transparency logs.

Index Terms—security, transparency protocols, logging

1. Introduction

Transparency logs are a core building block of many security-sensitive applications across domains such as encrypted messaging [1], [2], [3], [4], [5], Internet of Things (IoT) [6], encrypted backups [7], and code binary distribution [8], [9], [10], [11]. For example, incorrectly issued web certificates undermined the security of TLS traffic for many years because there was no way to verify if an attacker issued a web certificate for a website they did not control. Certificate transparency addressed this problem by providing transparency: the certificate transparency infrastructure deployed in browsers like Chrome and Safari does not prevent certificates from being incorrectly issued, but it does provide visibility into and consistency across all issued certificates, which can be used to detect misbehavior [12]. Key transparency solves a similar problem for

end-to-end encrypted messaging [1], [2], [3]: finding the correct public key for a user before sending an encrypted message. Key transparency allows clients to check that the server is advertising the correct key. If the server publishes the wrong key, the transparency log provides incontrovertible evidence of misbehavior.

A transparency log is, at its core, a *verifiable* key-value store. If a client writes a value, other clients querying the server should see the write. If the server is compromised, clients will either still read the correct value or obtain a proof of misbehavior. Transparency logs leverage the common client-server architecture with a single powerful server and many lightweight clients, and augment it by adding a small number of lightweight auditors to validate that the server correctly maintains system state. This makes transparency logs a useful tool for building verifiable key-value stores.

However, choosing a suitable transparency log for an application is not straightforward. Different transparency logs distribute computation and communication differently, leading to diverging performance and security properties. This forces a developer to sift through many transparency log designs to find one fitting her application. For example, specialized transparency log designs exist for web certificates [12], end-to-end encrypted messaging [1], [2], [3], IoT [6], code binary distribution [9], [10], [8], and encrypted backups [7], assuming different client, server, and auditor device profiles. To complicate matters further, the developer might need a transparency log where no specialized design exists for an application and workload. To achieve good performance, the developer must design a new log specific to the workload and the computational and communication constraints of different entities in their system. Identifying the correct transparency log or, worse, designing a new one, for every application is a massive barrier to adoption, especially for developers without expertise in transparency logs.

To address this problem, we introduce the concept of a *configurable transparency log*, a system that takes as input a workload description and the computational resources of each party and automatically outputs a transparency log design that matches the specification. We present the first configurable transparency log system: LegoLog. Using LegoLog, a developer can describe her application workload without additional expertise and automatically

NB: appendices, if any, did not benefit from peer review.

obtain a tailored log.

LegoLog’s design is guided by the observation that the vast majority of transparency log designs are built from the same key ingredients. Many designs are inspired by authenticated data structures [13], [14] and are built using data structures permitting efficient proofs that an item was appended to the log or that an item is included in the log. The variation in design then comes from how these ingredients are combined and how the work of verifying the transparency log state is distributed across different entities. We show how to combine these ingredients in a single generalized design that adjusts the underlying algorithm to meet application requirements.

To illustrate its generality, we show how LegoLog derives close approximations of existing, state-of-the-art transparency logs for several different workloads. For example, LegoLog can derive certificate transparency [12], binary transparency [15], multiple key transparency logs depending on application requirements—including CONIKS [1] and Merkle² [2]—and a transparency log for IoT [6].

The core challenges in designing LegoLog were threefold: (1) automatically generating a log design tailored to a workload, (2) creating an API for developers to easily specify workload details, and (3) exploring new design points representing different performance or security tradeoffs. When reasoning through a configurable design in LegoLog, we realized that parameters and tasks unnecessarily tied together in prior designs (e.g. update vs. monitoring intervals) could be decoupled. This allowed us to efficiently capture more workloads and, in some settings, to provide stronger security than existing transparency logs (e.g. preventing tampering vs. detecting misbehavior) as we explain below.

Decoupling the frequency of updates and monitoring (§3.3). In most existing transparency log designs, clients must come online to monitor for server misbehavior every time the server updates the log [1]. However, some applications require writes to propagate quickly with clients that are not regularly online (e.g., a user whose phone is stolen might want to update her messaging keys right away). We observe that we can decouple the update frequency and monitoring frequency. This insight allows us to explore new design points that allow for low update latency even if clients are not constantly online. In particular, we show how to trade off the frequency with which clients come online to perform checks for the cost of a lookup in the transparency log.

Allowing clients to detect misbehavior early (§3.3). In many existing transparency log designs [2], [1], [12], [15], [16], clients monitor for server misbehavior. If the server misbehaves, then the client obtains proof, but possibly only after an irreversible action has taken place due to an incorrect value (e.g., for a cryptocurrency directory or end-to-end encrypted messaging). In LegoLog, we explore a different security tradeoff by allowing clients who monitor regularly to detect server misbehavior *before* other clients use the incorrect values. This way, clients can not only detect server misbehavior, but also protect against its ill effects, provided that clients come online regularly. LegoLog achieves this while supporting low-latency updates, meeting both the performance and security requirements of an application.

Supporting offline clients (§3.4). In some applications, clients may disconnect for extended durations. When a client comes back online, it should be able to batch monitoring checks to perform them more cheaply. We show how to batch checks so that the client only has a small amount of work to do when it comes back online to detect misbehavior during the period it was offline. We draw inspiration from prior work on aggregating verification work [2]. We integrate these ideas in a way that is compatible with the other design parameters in our system without harming performance.

Designing an expressive developer API (§2.3). While LegoLog uses a set of internal system parameters to derive the correct transparency log construction, the developer has no intuition for how to set these parameters. We introduce an API that allows developers to specify the application workload and the computational and communication constraints of the clients and auditors. From that, the LegoLog’s planner outputs the number of cores and bandwidth required for the central server along with the internal system parameters for running LegoLog for the specified application (§4).

Evaluation summary (§5). To see how LegoLog generalizes to different applications, we empirically evaluate it end-to-end for three specialized transparency logs [1], [2], [15]. We also show that LegoLog can express six different applications, and we compare their asymptotic complexities to existing specialized transparency logs. We find that configurability does not come at the cost of performance: LegoLog can capture a variety of applications while performing comparably to existing, special-purpose transparency logs.

Limitations. While LegoLog captures a wide variety of transparency log settings, it does not capture those based on heavyweight cryptographic primitives, such as SNARKs [17], [18], [19], bilinear accumulators [20], or vector commitments [21]; these tools improve asymptotic performance, but generally have comparatively high overheads in practice. Also while we output the amount of server storage required, we do not optimize for it, as in the case of SEEMless [22] and Parakeet [3]; we leave the problem of optimizing for server storage to future work.

2. Overview

2.1. System architecture

In a LegoLog deployment (Figure 1), the central server provides an identifier-value store, and the clients and auditors enforce certain integrity guarantees on this store. We consider two types of clients: readers and writers. In some deployments (e.g. key transparency), clients are both readers and writers; in other deployments, clients perform only reads or only writes (e.g. in a producer-consumer model). Throughout the paper we refer to “identifier-value” pairs rather than “key-value” pairs to avoid confusion with cryptographic keys. We now describe the entities in a LegoLog deployment.

Readers. Lightweight clients that read identifier-value pairs.

Writers. Writers are clients that write identifier-value pairs. Writers monitor a subset of the identifiers (the identifiers

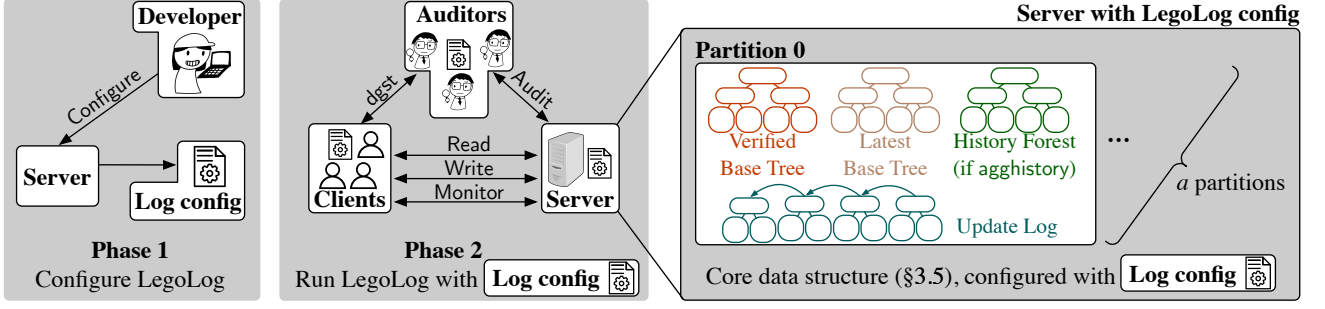


Figure 1: LegoLog overview. In Phase 1, the developer configures the LegoLog deployment for their application. The planner then generates a deployment configuration satisfying the requirements. In Phase 2, LegoLog runs with the configuration from Phase 1. Clients read, write, and monitor identifier-value pairs by interacting with the server. Auditors check the server correctly maintains the transparency log. The deployment configuration configures LegoLog’s core data structure (§3.5), and enables different possible performance characteristics.

they can write to) to ensure the server provides the correct value to readers for any identifier in the writer’s set. Writers should be lightweight and may go offline for some time.

Auditors. Auditors check that the server maintains the transparency log correctly. Auditors also service client requests for the log digest, allowing clients to ensure that the server answers queries relative to the auditors’ view of the log.

Server. The server maintains the transparency log and services read and write requests from clients. The server also sends data to the auditors that allow the auditors to ensure that the server is maintaining the log correctly.

2.2. System API

We now describe the API that LegoLog provides.

Readers can execute the following API call with the server:

- $\text{Read}(\text{config}, \text{dgst}, \text{id}, t) \rightarrow \{\text{val}, \perp\}$: To read the value of id relative to a digest dgst at epoch t for a deployment with configuration config , the reader queries the central server for the value val and a proof. If the proof verifies relative to the auditors’ digest, the reader outputs val ; otherwise, it outputs $\perp$.

Writers then execute the following API calls with the server, as writers need to write values to identifiers, and then monitor the value of the identifiers they have write privileges for:

- $\text{Write}(\text{config}, \text{sk}, \text{id}, \text{val}, t) \rightarrow \{0, 1\}$: To write an (id, val) pair in epoch t for a deployment with configuration config , the writer sends a request to the central server. The server updates its state and the client outputs “1” if (id, val) was correctly included and “0” otherwise. If this is the first write to id , the server associates id with the public key corresponding to sk . If this is not the first write to id , the write is only successful if sk matches the secret key used on previous successful writes to id . As in prior work [2], this secret key ensures that only the owner of the secret key can update the value corresponding to id .
- $\text{Monitor}(\text{config}, \text{dgst}, \text{id}, \text{val}, t) \rightarrow \{0, 1\}$: To verify that id still maps to val relative to digest dgst at epoch t in a deployment with configuration config , the writer queries the central server for a proof. If the proof verifies relative to the auditors’ digest and asserts that id maps

to val , then the writer outputs “1”; otherwise, it outputs “0”.

Auditors also run the following algorithm periodically:

- $\text{Audit}(\text{config}, \text{dgst}_{\text{old}}, \text{dgst}_{\text{new}}, \pi) \rightarrow \{0, 1\}$: The auditor takes in configuration config , an old digest dgst_{old} , a new digest dgst_{new} , and a proof π from the central server. The auditor outputs “1” if π certifies that dgst_{new} correctly extends dgst_{old} , and “0” otherwise.

2.3. Developer API

LegoLog exposes the following API to developers:

- $\text{Configure}(\text{workload}, \text{reader}, \text{writer}, \text{auditor}) \rightarrow (\text{server}, \text{config})$: Given a description of the workload, the reader specification reader , the writer specification writer , and the auditor specification auditor , output a server specification server , as well as the system parameters config that are used by the readers, writers, server, and auditors.

LegoLog allows a developer to set parameters to derive a transparency log suited to her workload (Table 1 lists the parameters). Given a workload specification, LegoLog outputs the necessary server capacity and the configuration.

If a developer makes a mistake when specifying the workload or there is an unexpected change in the workload or client connectivity, LegoLog still provides the standard transparency log security guarantees, but the performance and additional security of active prevention may be affected. For example, performance could be affected by incorrect server requirements or a suboptimal update period or number of partitions. Security could be affected if clients cannot come online every verification period: in this case, they can only detect server misbehavior after the fact.

2.3.1. Example usage: Key transparency

To build intuition for how a developer would characterize a workload, we take key transparency as a running example [1], [23]. In this application, the transparency log holds a mapping of usernames to public keys (e.g. for end-to-end encrypted messaging), and clients monitor the log to ensure that the mapping of usernames to public keys is maintained correctly.

Say that a developer wants to configure LegoLog for key transparency in Signal. We now show an example of

TABLE 1: Inputs and outputs to the LegoLog configuration algorithm exposed to the developer.

INPUTS		
Workload	N	Number of users
	t_W	Max time for writes to propagate (s)
	t_R	Max time for reads to be processed (s)
	burst_R	Max number of reads that can be handled immediately in t_R (reads/ t_R)
	burst_W	Max number of writes that can be handled immediately in t_W (writes/ t_W)
Read	bw_R	Reader bw utilization (bytes/hr)
	cores_R	Reader CPU utilization (% of CPU cycles, averaged over 1 hour)
Write	bw_W	Writer BW utilization (bytes/hr)
	cores_W	Writer CPU utilization (% of CPU cycles, averaged over 1 hour)
	t_{refresh}	Background refresh rate, frequency at which writer queries server (s)
	t_{offline}	Max writer offline time staying in comm and utilization bounds (s)
Audit	bw_A	Auditor-server BW (bytes/hr)
	cores_A	Number of auditor cores
OUTPUTS		
Server	bw_S	Server BW utilization (bytes/s)
	cores_S	Number of server cores
	storage_S	Server storage (bytes)
Sys params	t_v	Length of verification period (s)
	t_u	Length of update period (s)
	a	Number of partitions
	verifier	Party responsible for verification period checks (client or auditor)
	agghistory	Whether or not to aggregate base trees for efficient delayed verification (T/F)

how the developer can specify the workload to LegoLog. Signal reportedly has roughly 40 million active users as of January 2021 [24], with roughly 125 million downloads in 2021 for an average of 342K downloads a day [25]. Therefore we can set the number of users $N = 125\text{M}$. When a user moves to a new device, it is important for security to update their public key quickly, and so we can set the maximum time for writes to propagate t_W to 15 seconds. Reads must be processed quickly so that users can message other users right after downloading Signal or importing a new contact, and so we set t_R to 30 seconds (it will take the user some time to type a message before trying to send). The number of downloads is roughly equivalent to the number of key updates (i.e. the number of writes). The average number of key updates every 15 seconds is 59, and so we set our application to handle up to a burst of up to 75 writes every 15 seconds by setting $\text{burst}_W = 75$. The choice of how large of a burst to accommodate is somewhat arbitrary and depends on how much the developer wants to minimize the cost of the service vs. how important write latency is in the event of a burst (if there are more requests than can fit in the burst, the client can simply try again later).

The average Android user has roughly 5,000 contacts synced to their device [26], and so we conservatively assume that every contact uses Signal and that users need to query the transparency log for a user’s public key roughly once a year (assuming that users buy new phones roughly once a year). This results in 200B read requests over a year, or on average 190K read requests every 30 seconds. If we want to accommodate a burst of 300K read requests every 30 seconds (again, the choice of burst relative to the average number of requests is somewhat arbitrary), we set

$\text{burst}_R = 300,000$.

In key transparency, every client is both a reader and a writer (clients read public keys for their contacts, and write their own public key). We can then set the reader and writer parameters in the same way based on the acceptable communication and computation costs for LegoLog in a Signal deployment. We use these communication and computation constraints to set the max bandwidth bw_R and bw_W , CPU utilization cores_R and cores_W , and background refresh rate t_{refresh} . Based on how long clients typically go offline, we can set the max offline time t_{offline} . We also provide as input the computation and communication that the auditors can devote to LegoLog (cores_A and bw_A , respectively). Based on these inputs, LegoLog outputs the required server bandwidth (bw_S), number of cores (cores_S), and storage (storage_S), as well as the internal parameters for running the system.

2.4. Security guarantees

We now describe the security guarantees that LegoLog provides. Like other transparency logs, LegoLog defends against a malicious attacker that has compromised the server, any number of clients, and all but one auditor that can communicate with the clients and the server. Against such an attacker, LegoLog ensures that the client can *detect* if the attacker tampered with any (id, val) pairs the client is monitoring.

Furthermore, if the client comes online at regular intervals, the client can *prevent* this tampering. Transparency logs traditionally only allow a client to detect when the server has misbehaved. In LegoLog, if the client comes online periodically, the client can detect if the server will

advertise the wrong value for a given identifier before any clients use this incorrect value. In this event, the client can report this misbehavior before the wrong values are used, which is valuable for applications where clients take irreversible actions based on values in transparency logs (e.g. by sending cryptocurrency transactions or sending sensitive information via encrypted email or messaging). Note that the client can only report this misbehavior if it has access at the time to some public forum of sorts where it can notify other clients of the server’s misbehavior in a timely manner.

Like other transparency logs [1], [2], [22], we do not guarantee availability: the server can refuse to provide service. Some auditors can refuse to provide service, but as long as clients can continue contacting at least one honest auditor, they can make requests and have strong integrity guarantees.

If a client does not monitor its identifier-value pairs, there are no integrity guarantees for this client’s data. Also, a malicious client (who could collude with the server) cannot affect the identifier-value pairs of an honest client.

Security game. We define security for LegoLog via a security game (Experiment 6 in Appendix C). At a high level, the challenger plays the role of honest clients and an honest auditor. The adversary chooses the identifier-value pairs that the clients can write and monitor each round. At the end of the game, the challenger monitors and then reads all identifier-value pairs that it has written. The adversary wins if all auditing and monitoring checks pass while ensuring one of the reads at the end of the game outputs a value that does not match the client’s latest write. This game captures log tampering detection and prevention. For detection, the adversary can choose which identifier-value pairs are not monitored and the challenger must still detect any tampering when it monitors all identifier-value pairs at the end of the game. For prevention, the client does not monitor after its last read at the end of the game, so any misbehavior must be caught by prior monitoring or auditing checks. Note that the game also captures correctness as client writes and reads values according to a reference dictionary.

Definition 1. Let Π be a configurable transparency log protocol and $\mathcal{C}$ be the set of all possible configurations. Then we say that a configurable transparency log protocol Π is secure if for all non-uniform PPT adversaries $\mathcal{A}$ in Experiment 6 with advantage TLogAdv and for all possible configurations $\text{config} \in \mathcal{C}$ with sufficiently large security parameter λ ,

$$\text{TLogAdv}[\mathcal{A}, \Pi](\text{config}) \leq \text{negl}(\lambda)$$

Theorem 2. *When instantiated with a signature scheme that provides existential unforgeability under chosen-message attack (EUF-CMA) and a collision-resistant hash function, LegoLog is secure according to Definition 1.*

Proof sketch. We must reduce the advantage of the adversary $\mathcal{A}$ in our security game to the advantage of an adversary against an EUF-CMA signature scheme or of an adversary against the collision-resistant hash function. We prove this by first constructing a hybrid game where the adversary wins if the challenger outputs an old identifier-value pair as a result of the reads at the end of the game,

and prove that the difference in adversary advantage $\mathcal{A}_\Delta$ between the two games is negligible when using a secure signature scheme by constructing an adversary $\mathcal{B}$ that uses $\mathcal{A}_\Delta$ to win in the EUF-CMA game. We then prove that an adversary $\mathcal{A}$ that wins this hybrid game can be used to construct an adversary $\mathcal{C}$ that can find a collision in the hash function. We include a full proof in Appendix C.

3. System design

3.1. Building blocks

Merkle trees. LegoLog uses binary Merkle trees [27], which are hash-based data structures that enable efficient verification of contents in the tree. Each leaf node contains a hash of the data it holds, and every non-leaf node contains a hash of its children. In transparency logs, it is common for the leaves of a Merkle tree to be either organized in prefix (lexicographic) order or chronological (time of append) order.

Prefix Merkle trees organize their leaves in lexicographic order. This organization enables efficient lookups because a client knows the exact leaf to request an inclusion proof for when looking up an identifier-value pair. An example prefix Merkle tree and inclusion proof (shaded in orange) is depicted in Figure 2. A proof of exclusion for some j is a proof of inclusion to the longest-prefix match of j currently in the tree. That node will either be a leaf node $i \neq j$ that matches the first few bits of j or an empty node that is a prefix of j [1]. While prefix Merkle trees provide efficient lookups, it is inefficient to prove that a newer prefix tree did not remove any contents of the old one. In existing transparency logs based on prefix Merkle trees, clients must monitor their identifiers every epoch to check for server equivocation [1].

Chronological Merkle trees have leaves organized by time the leaf was appended to the tree. Unlike prefix Merkle trees, chronological Merkle trees have inefficient lookups because a client may need to search through all the leaves to find a given identifier-value pair. Figure 3 shows a chronological tree at two points in time. It is efficient to prove that a chronological Merkle tree extends an older tree (i.e., is append-only). In Figure 3, the extension proof consists of the nodes shaded in blue. A detailed algorithm for generating the extension proof is found in §2.1.4.1. of [12].

3.2. Sharding chronological trees

Chronological and prefix Merkle trees have complementary properties that are both desirable. Chronological trees have efficient extension proofs that prove the append-only property of the tree [12], but do not support efficient lookups. On the other hand, prefix trees support efficient lookups but require users to monitor their identifier-value pairs every epoch to ensure that the server has not equivocated [1]. Ideally, we would like a system that supports *both* efficient lookups and extension proofs so that users do not have to monitor their own identifier-value pairs every epoch.

Chronological trees already support efficient extension proofs and do not require clients to monitor their key every

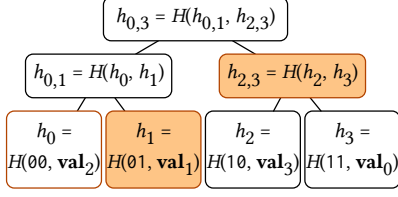


Figure 2: An example prefix Merkle tree containing 4 leaf nodes in lexicographic order. $H(\cdot)$ is a hash function and val_i is a node added at time i . An inclusion proof for the identifier-value pair $(00, \text{val}_2)$ consists of the nodes shaded in orange, which are sufficient to reconstruct the root hash $h_{0,3}$.

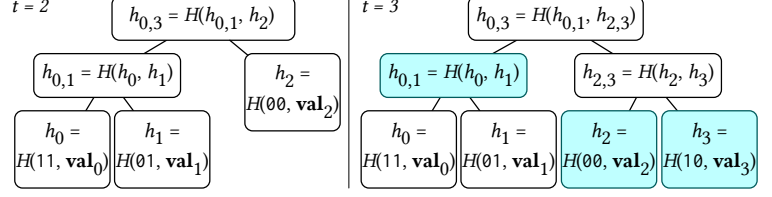


Figure 3: An example chronological Merkle tree that contains 3 leaf nodes at time $t = 2$ and 4 leaf nodes at time $t = 3$. $H(\cdot)$ is a hash function and val_i is a node added at time i . The leaf nodes are ordered by the time the node was added. The extension proof to prove that the $t = 3$ tree extends the $t = 2$ tree consists of the nodes shaded in blue, which commit to the same values in the $t = 2$ tree and are sufficient to calculate the root hash $h_{0,3}$ in the $t = 3$ tree.

epoch. Due to its leaves being ordered by append time, a lookup requires the client to look through all the leaves in the tree. We observe that we can reduce the cost of lookups by reducing the size of the chronological tree by sharding identifier-value pairs across multiple chronological trees (Figure 4). A lookup for id would entail hashing id to find its corresponding chronological tree and linearly scanning through all of its leaves for relevant updates to id . Adding another chronological tree shard means that auditors have another extension proof to check every epoch.

If u is the number of updates to identifier-value pairs in the transparency log, a is the number of partitions, and l is the number of lookups/epoch, the work done by an auditor is equal to the work of checking an extension proof times the number of chronological trees of size u/a :

$$\text{work}_A = a \max(1, \log_2(u/a))$$

and the work done by clients for lookups is equal to the number of lookups times the work of checking each leaf node in the chronological tree of a partition (size u/a):

$$\text{work}_C = lu/a$$

The total work is the sum of auditor and lookup work:

$$\text{work}_T = a \max(1, \log_2(u/a)) + lu/a$$

Sharding chronological trees provides a configurable system parameter—the number of partitions—that affects client and auditor work. This parameter can be tuned for different objectives. For example, the optimal number of shards that minimizes total work can be found by taking the partial derivative $\partial/\partial a$ of work_T and solving for its roots. By sharding chronological trees, we are a step closer to achieving a system where clients do not have to monitor their keys (due to efficient chronological tree extension proofs) and where lookups are efficient (requiring u/a rather than u work).

3.3. Compacting chronological trees

As time goes on and the number of updates increases, client lookup work continues to increase because a lookup entails a linear scan through all the leaves in the chronological tree. In order to reduce lookup time, we can routinely roll up the updates into a prefix tree after some time. We call this prefix tree the *base tree* and the chronological tree holding updates the *update log*. In each partition, LegoLog

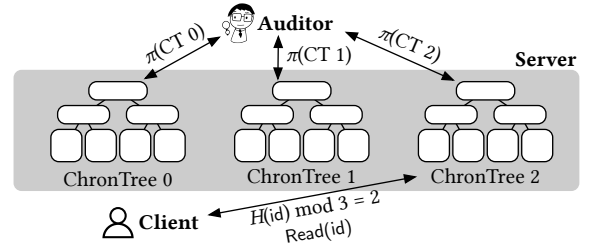


Figure 4: Sharding chronological trees. When looking up an id , the client calculates the partition the id belongs to and looks up id in the corresponding chronological tree. Auditors are responsible for verifying extension proofs for all of the chronological trees.

keeps a core data structure (CDS) consisting of a base tree and an update log.

The client must now periodically monitor its own identifier-value pairs and ensure that its updates are properly incorporated into the latest base tree. Our key insight is decoupling when a value is updated and when a value should be monitored (verified) by the client. The frequency the client needs to monitor its values is controlled by the *verification period* t_v and the update propagation time is controlled by the *update epoch* t_u . During each verification period t , the client verifies its values in the latest base tree. The previous base tree, verified during period $t - 1$, and the update log containing updates since period $t - 1$ are used to look up values during verification period t . A verification epoch can contain multiple update epochs: longer verification periods correspond to more update epochs ($\uparrow$ lookup work) and less frequent monitoring ($\downarrow$ monitoring work).

Update log optimization. To further decrease lookup cost, the server can roll its updates every update epoch into a prefix tree and append the root hash of the prefix tree to the update log. If the client made an update, the client should check that its update appears in the prefix tree for that update epoch. A lookup for id will consist of 1) looking up id in the base tree and 2) looking up id in every update epoch's prefix tree. Figure 5 shows a server partition that compacts chronological trees and optimizes the update log.

Preventing tampering vs. detecting misbehavior. Clients look up values in the base tree from the previous verifica-

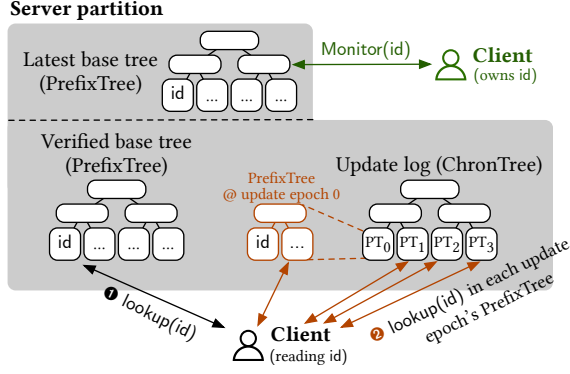


Figure 5: A server partition that compacts chronological trees contains a verified base tree, the latest base tree, and an update log. A base tree is a prefix tree that routinely rolls up updates. The update log tracks changes after the verified base tree, and is a chronological tree of the root hashes of prefix trees for each update epoch. A client looks up id ① in the verified base tree and ② in each update epoch’s tree. The client that owns id verifies that the latest base tree contains the correct value.

tion period and the tree in the current verification period can be thought of as being in ‘staging’. If the client comes online periodically, the client can detect if the server will advertise the wrong value for a given identifier before any clients use this incorrect value. In this event, the client can report this misbehavior before the wrong values are used. Depending on the security guarantees and performance properties that the developer desires for her application, the planner will set t_v accordingly (§4.2) to achieve the desired configuration.

3.4. Supporting offline clients

Ideally, clients come online every verification period and monitor to detect server equivocation. However, in workloads with clients that are offline for multiple verification periods, clients will need to check the base tree in every verification period it was offline for. To support *efficient* delayed verification for clients that are offline for multiple verification periods, we take inspiration from the ideas of Merkle² [2] and show how to incorporate them into LegoLog, which is general-purpose and configurable for different workloads.

History forest. We construct a chronological forest of base trees, which we call a *history forest*. This data structure is related to the chronological forest in Merkle² [2] where leaves are identifier-value pairs arranged in historical order and each intermediate node contains a prefix tree representing the subtree rooted at that node. Our data structure differs in that the leaves are base trees instead of identifier-value pairs and intermediate nodes contain the most recent base tree of their child nodes (i.e., the right child node). Nodes are added to the forest in an immutable manner; once added, a node cannot be changed. As a result, intermediate nodes can only be created when their subtree is full, so the data structure is a *forest* of trees rather than a single tree. Figure 6 shows an example history forest with 7 base trees (7 verification periods) and 3 history

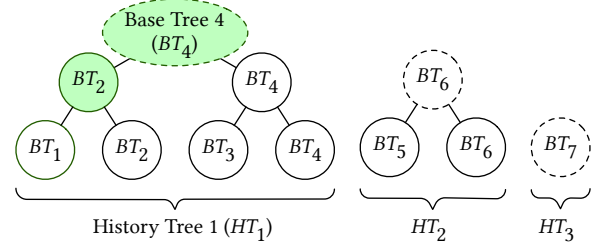


Figure 6: A history forest of base trees from 7 verification periods. There are 3 history trees, and each history tree’s leaves are arranged in chronological order. A monitoring proof for an update in BT_1 consists of the nodes shaded in green. A lookup consists of looking up a value in each history tree’s root (nodes with dotted outline). The lookup and monitoring proof both include the root (BT_4) of HT_1 .

trees. Adding the next base tree BT_8 to this history forest first involves creating a leaf node representing BT_8 . Then, an intermediate node can be created as the parent of the leaf nodes BT_7 and BT_8 . This results in HT_3 being a full history tree with 2 leaves. Because HT_2 is also a full history tree with 2 leaves, we can create an intermediate node as the parent of HT_2 and HT_3 , effectively merging HT_2 and HT_3 into one full history tree HT'_2 with 4 leaves. Finally, since HT_1 is a full history tree with 4 leaves, we can create an intermediate node as the parent of HT_1 and HT'_2 , resulting in a full history tree with 8 leaves where the root contains BT_8 . Note that adding BT_8 did not modify any existing nodes.

A monitoring proof in the history forest during verification period t_m for update u made during verification period t_u consists of the path from t_u to the history tree root containing t_u in the history forest at time t_m . For each node in the path, the client looks up the value of u in the corresponding base tree. Because the history forest is append-only and its nodes do not change after being added, the client need only verify a given verification period’s base tree once. Figure 6 shows a monitoring proof consisting of nodes (green) for an update made during verification period 1 (corresponding to BT_1) and monitoring during verification period 7. The client ensures that the update has propagated to BT_1 , BT_2 and BT_4 .

The monitoring proof alone is insufficient to detect whether the server has removed an update, as it does not check every prefix tree. For example, in Figure 6, the monitoring proof for an update made in BT_1 does not check the base trees BT_3 and BT_5-7 . Instead, during a *lookup*, our system must ensure that at least one base tree that the client looks up overlaps with a base tree verified in a monitoring proof. LegoLog achieves this by having the client look up the value in *every* history tree’s root base tree. Figure 6 shows a lookup during verification period 7 consisting of each history tree’s root (nodes with dotted outline): BT_4 , BT_6 , and BT_7 . Observe that BT_4 overlaps with the monitoring proof (nodes shaded in green), so the client can detect that the server removed the update if it is present in BT_4 but missing in BT_6 or BT_7 .

Our history forest data structure lends itself to an efficient monitoring proof for clients that are offline for multiple verification periods and trades off a higher lookup cost for this increased monitoring efficiency. A similar

$\log n$ (where n is the number of verification periods) checking scheme is proposed in a design draft of Google’s new KT system [23], but mapping the version checks to checking a path in the history tree is not stated explicitly in the document.

The optionality of using a history forest allows LegoLog to be configured for different workloads. For a workload where clients are offline for long periods of time, LegoLog can be configured to use the history forest, resulting in a lookup cost of $O(\log^2 n)$ and a monitoring cost of $O(\log^2 n)$. However, for a workload where clients are regularly online, LegoLog can be configured to not use the history forest, resulting in a lookup cost of $O(\log n)$ and a monitoring cost of $O(\log n)$.

Signing updates. While monitoring proofs prevent the server from omitting values, they do not prevent it from maliciously inserting values. To prevent this, we use signatures, a solution inspired by previous work [2]. For a given id , and verification period t , the writer first registers a verifying key on the log. In order to ensure that this is the first time a verifying key is being registered for this id , the writer must verify a proof that id is not present in the base tree at verification period $t - 1$ (BT_{t-1}). Upon verifying this proof, the writer inserts the verifying key for id into the log. Then, for each update to id , the writer attaches a signature. When reading id , the client can look up the verifying key in the base tree and use it to verify the signature.

3.5. Putting it together: Core data structure

LegoLog partitions identifiers and in each partition, uses a configurable core data structure (CDS). A CDS is comprised of a digest for the previous verification period (forming a hash chain between verification periods) and *three* Merkle trees (Figure 5): (1) a verified based tree, (2) the latest base tree (that is being verified), and (3) an update log. Base trees are prefix Merkle trees mapping identifiers to values. The update log is a chronological Merkle tree tracking updates to the verified base tree. The leaves of the update log are prefix Merkle trees containing the updates of an update epoch.

LegoLog’s CDS can be configured to support efficient delayed verification by aggregating the base trees into the history forest described in §3.4. When the *agghistory* parameter is true, LegoLog’s CDS contains a history forest of base trees instead of a verified base tree and the latest base tree.

3.6. LegoLog protocol

We now describe the LegoLog protocol. We include a full description of the protocol in §B.

Reading. In order to read the value of id relative to a digest dgst obtained from an auditor, ❶ the client first computes the partition for the queried id : $H(\text{id}) \bmod a$. ❷ Then, the client retrieves the root of the verified base tree for that partition from dgst . ❸ If the client does not have the verifying key for id , it first looks up the verifying key in the verified base tree (via inclusion proofs). ❹ The client then looks up id in the verified base tree (via inclusion proofs). ❺ For each update epoch since the start of the verification

period, the client receives an inclusion or exclusion proof for id in the update log from the server. The client can retrieve exclusion proofs from the most recent update epochs until hit the most recent inclusion proof, or get all of the exclusion proofs if it wants the full history of changes to id . ❻ The client outputs (id, val) where val is the latest value with a signature that successfully verifies with the verification key.

History forest case. If *agghistory* = true, the client follows ❶–❸ as previously described, except in ❷, instead of retrieving a single base tree root, it gets the base prefix tree corresponding to the root of each history tree in the history forest. ❹ The client then looks up id in each base prefix tree (via inclusion proofs). The client follows ❺–❻ as previously described.

Writing. To update a value, the client ❶ signs the update with its signing key and sends the (id, val) pair and signature to the server. ❷ The server appends to the (id, val) pair to the partition (determined by $H(\text{id}) \bmod a$). ❸ The client then checks that the update was correctly incorporated into the update log in the next update epoch (via inclusion proofs).

Monitoring. We describe the cases where the client or the auditor performs monitoring.

Clients perform monitoring. If the client is responsible for monitoring values it has written (verifier = client), ideally the client comes online every verification period, finds the partition corresponding to the id it wants to monitor by computing $H(\text{id}) \bmod a$, and queries the latest base tree in that partition to make sure the value is correct. Clients have until the next verification period—when the latest base tree becomes the next verified base tree—to check this value.

History forest case. If *agghistory* = true, then when the client comes online, ❶ it queries the auditors for the roots of the history forest in the partition corresponding to id . ❷ In each update period where the client made an update to id , it retrieves an inclusion proof from the LegoLog server of the base tree root for each node on the path from the update epoch to the root in the corresponding history tree. An example path is shown in Figure 6. ❸ For each base tree root the client receives that the client has not checked in the past, the client will check the value of id in that tree by requesting an inclusion proof from the server.

Auditors perform monitoring. If the auditor is responsible for monitoring values on behalf of the users (verifier = auditor), then in each verification period, the auditors check that the latest base tree in each partition corresponds to the contents of the update log applied to the previous base tree.

Auditing. In each update epoch, the auditors check that the update log in each partition is append-only by requesting extension proofs from the server (a total update logs are checked). In each verification period t_v , the auditors keep track of the digest, check it amongst themselves, and serve it to clients.

History forest case. If *agghistory* = true, auditors check the history forest of base trees in each partition is append-only.

4. LegoLog Planner

LegoLog’s configurability comes from an automatic planner that generates the transparency log design for a given application workload description. The planner takes as input a workload description and specifications of the readers, writers, and auditor and outputs internal system parameters and a central server specification (Table 1). By automatically tuning the internal system parameters, it can customize the log for the specified performance and security needs.

4.1. Specifying entities

Requiring the developer to directly specify device properties (e.g., CPU, target CPU utilization, network bandwidth) can be difficult or insufficient for determining the optimal configuration. For example, the developer may not know a mobile client’s exact CPU utilization because battery life is often the priority [28]. They often want to optimize for an objective more complex than a single target metric. Sometimes this means minimizing client-side CPU utilization for mobile clients; sometimes it means reducing total system workload.

Instead, we allow developers to specify the *type* of device that the readers, writers, and auditors run on. The device type translates to a heuristic about the device’s properties and a corresponding optimization objective. For example, if a developer specifies that readers run on mobile phones, the planner will minimize client-side work.

The planner must also know the auditor’s compute budget. It determines this by running a benchmark on a representative auditor device, which measures the device hash rate. This information and the developer’s target auditor CPU utilization sets a constraint on the maximum auditor work.

4.2. Determining system parameters

We now describe how LegoLog’s planner maps developer-specified parameters to internal system parameters (Table 1).

Update period. The update period t_u directly corresponds to t_W , the maximum time for writes to propagate.

Verification period. The verification period t_v is lower-bounded by the update period t_u and upper-bounded by t_{refresh} , the background refresh rate of writers t_{refresh} (i.e., the frequency at which writers query the server). In order to find the optimal t_v for a fixed number of partitions a , we need to consider all possible verification periods t_v where $t_u \leq t_v \leq t_{\text{refresh}}$. The cost of a lookup (work_L) without the history forest ($\text{agghistory} = \text{false}$) is defined in terms of the number of hashes the client needs to compute in order to verify the update log (work_{UL}) and the base tree:

$$\begin{aligned}\text{work}_{UL}(t_v, a) &:= t_v/t_u (\max(1, \lceil \log_2(\text{burst}_W/a) \rceil)) \\ \text{work}_L(t_v, a) &:= \text{work}_{UL} + \max(1, \lceil \log_2(N/a) \rceil)\end{aligned}$$

The cost of a lookup when $\text{agghistory} = \text{true}$ adds a $\log_2(N)$ factor because the client looks up id in multiple base trees:

$$\text{work}_{L, \text{aghs}}(t_v, a) := \text{work}_{UL} + \log_2(N) (\max(1, \lceil \log_2(N/a) \rceil))$$

The cost of monitoring without the history forest is the cost of doing lookups in the base tree for every verification period missed while the client was offline:

$$\text{work}_M(t_v, a) := \max(1, \lceil \log_2(N/a) \rceil) \cdot \frac{t_v}{t_{\text{offline}}} \cdot \frac{N}{t_{\text{offline}}}$$

When $\text{agghistory} = \text{true}$, the cost of monitoring is much lower because of the history forest:

$$\text{work}_{M, \text{aghs}}(t_v, a) := \max(1, \lceil \log_2(N/a) \rceil) \cdot \frac{N}{t_{\text{offline}}}$$

Using the above equations, we can compute the optimal t_v^* for a fixed number of partitions a :

$$\begin{aligned}\text{work}_C(t_v, a) &= \text{work}_L(t_v, a) + \text{work}_M(t_v, a) \\ \text{work}_{C, \text{aghs}}(t_v, a) &= \text{work}_{L, \text{aghs}}(t_v, a) + \text{work}_{M, \text{aghs}}(t_v, a) \\ t_v^*(a) &:= \arg \min_{t_v \in [t_u, t_{\text{refresh}}]} \text{work}_C(t_v, a) \\ t_v^*(a, \text{aghs}) &:= \arg \min_{t_v \in [t_u, t_{\text{refresh}}]} \text{work}_{C, \text{aghs}}(t_v, a)\end{aligned}$$

Number of partitions. In order to set the number of partitions a , we need to consider all possible numbers of partitions a where $a \leq N$. The auditor work when $\text{agghistory} = \text{false}$ consists of verifying the append-only property of the update log and keeping track of the base tree root:

$$\begin{aligned}\text{work}_{A0UL}(a) &:= \max(1, \lceil \log_2(t_v^*(a)/t_u) \rceil) \\ \text{work}_A(a) &:= a \cdot (\text{work}_{A0UL}(a) + 1)\end{aligned}$$

When $\text{agghistory} = \text{true}$, auditors must verify the append-only property of the update log and the history tree:

$$\text{work}_{A, \text{aghs}}(a) := a \cdot (\text{work}_{A0UL}(a) + \log_2(N))$$

Note that total auditor work monotonically increases with a . The optimal a^* is one that satisfies the auditor’s CPU utilization constraint cpu_A while minimizing client work:

$$\begin{aligned}a^* &:= \arg \min_{a \in [1, N]} (\text{work}_C(t_v^*(a))) \quad \text{s.t. } \text{work}_A(a) \leq \text{cpu}_A \\ a^*(\text{aghs}) &:= \arg \min_{a \in [1, N]} (\text{work}_{C, \text{aghs}}(t_v^*(a, \text{aghs}))) \\ &\quad \text{s.t. } \text{work}_A(a) \leq \text{cpu}_A\end{aligned}$$

Using history forest. After computing the optimal number of partitions and verification period length for $\text{agghistory} = \text{true}$ and false , we can determine whether or not to use the history forest by comparing the client work of both options:

$$\text{agghistory} := \text{work}_{C, \text{aghs}}(t_v^*(a^*, \text{aghs})) \stackrel{?}{<} \text{work}_C(t_v^*(a^*))$$

Verifier. The planner looks at the leftover auditor CPU cycles after computing the above costs. If the remaining

auditor cycles is enough to monitor on behalf of all writers, then the planner sets verifier = auditor (otherwise verifier = client).

Option for detecting misbehavior early. LegoLog’s planner can also help decide whether to detect misbehavior *before* other clients use incorrect values. This can be achieved by setting $t_v = t_{\text{offline}}$ so that users never go offline longer than a verification period. The planner computes the optimal configuration for this setting. The developer can then compare this to detecting misbehavior reactively (instead of preventing it) and decide which approach to use for her application.

Server requirements. The server requirements returned by the planner are the number of hashes/second that the server needs to retrieve and transmit to the clients and auditors. This is computed by computing work_C and work_A for the optimal a^* and t_v^* , while accounting for burst_R .

Network bandwidth. Meeting network bandwidth requirements follows the same logic of meeting CPU requirements since network usage $\text{bw}(\cdot)$ depends on the number of hash computations for lookup, monitoring, and auditing. If network bandwidth is the bottleneck, the planner optimizes over $\text{bw}(\text{work}_C)$ and $\text{bw}(\text{work}_A)$, and uses nw_A as the constraint.

5. Evaluation

In our evaluation, we aim to answer three questions:

- 1) How do parameter settings affect performance? (§5.2)
- 2) Does LegoLog perform comparably to existing special-purpose logs for different applications? (§5.3)
- 3) What workloads can LegoLog’s API express, and how does it perform for these? (§5.4)

In answering these questions, we show that LegoLog’s configurability does not come at a substantial loss in performance: LegoLog performs comparably to existing specialized transparency logs tailored to individual applications.

5.1. Implementation

Our LegoLog implementation is a fork of the existing Merkle² [2] implementation using Go and Python. We added ~ 4000 LOC in order to implement additional core data structures, the LegoLog protocol, and the LegoLog planner. We use gRPC v1.49 for communication and SHAKE-128 as our hash function. We release LegoLog as an open-source project at <https://github.com/vivi/legolog>.

Memory optimization. To avoid copying the entire base tree every verification period, we use a combination of fat nodes and path copying from persistent data structures [29]. We maintain the invariant that child nodes are always at least as old as their parent nodes, which enables constant-time access to nodes at any verification period. We further optimize by only path copying between verification periods.

Experiment setup. We ran experiments on Google Cloud. For the server, we used an n2-highmem-32 instance (32-core Intel Xeon CPU, 256GB RAM). For the client, we used an n2-standard-8 instance (8-core Intel Xeon CPU, 32GB RAM). For the auditor, we used a lightweight

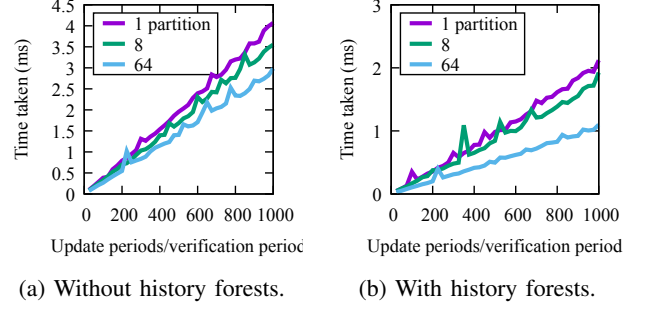


Figure 7: Time taken for the server to generate a lookup proof for different settings of update periods in a single verification period (t_v/t_u) and number of partitions a .

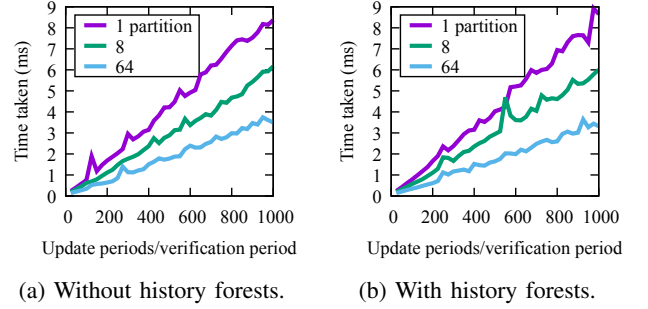


Figure 8: Time taken for the client to validate a lookup proof for varying t_v/t_u and number of partitions a .

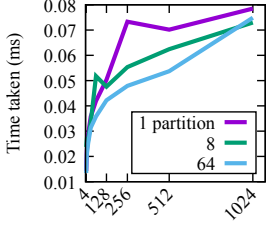
n2-standard-2 instance (2-core Intel Xeon CPU, 8GB RAM). For benchmarks, we read and write identifiers uniformly at random.

5.2. Microbenchmarks

We first investigate how different settings of internal system parameters impact the performance of LegoLog.

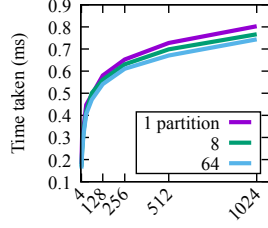
Read cost. We measure the client and server overheads of reading a value (Figures 7 and 8) for different numbers of partitions and different ratios of update periods to verification periods. We generate these plots for varying numbers of update periods in a single verification period (t_v/t_u) where each update period has 200 appends. The server is preloaded with 1M entries. Increasing the number of partitions decreases both client and server work because having more partitions reduces the amount of data in one partition and so the overhead of proving and verifying is smaller. Increasing the ratio of update periods to verification periods makes it possible for client writes to propagate without clients having to run monitoring as frequently. However, increasing this ratio also increases client and server lookup work. This increase is expected, as a lookup proof is required for each update period that has elapsed since the end of the last verification period.

Figures 9 and 10 measure the time taken by LegoLog configured to use history forests (`agghistory = true`) to generate and validate a lookup proof respectively, as the number of verification periods elapsed increases. The amount of work increases logarithmically with the number of verification periods elapsed because a lookup when `agghistory = true` consists of a lookup for that value in



Verification periods elapsed

Figure 9: Lookup proof generation work over multiple verif. periods ($t_v = t_u$).



Verification periods elapsed

Figure 10: Lookup proof validation time over multiple verif. periods ($t_v = t_u$).

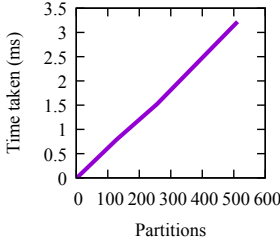


Figure 11: Median auditor work between update epochs as partitions a increase.

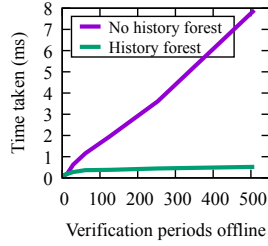


Figure 12: Client monitoring work as offline time increases.

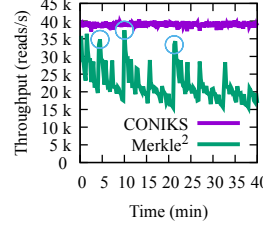
each history tree’s root and there are at most $\log(n)$ history trees in the history forest, where n is the number of nodes in the history forest (i.e., number of verification periods).

Auditor overhead. Figure 11 shows how the number of partitions affects auditing time. For each partition, the auditor must verify a chronological tree extension proof, and so the auditor work grows linearly with the number of partitions.

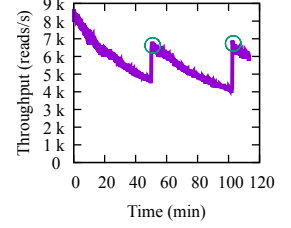
Monitoring overhead. For client monitoring, we evaluate the case in which the monitor is offline for some number of verification periods (Figure 12). We measure the time taken to verify that a particular identifier-value pair exists in each verification period. Without the history forest, this means verifying the existence proof for each base verification tree. With the history forest, the monitor only has to verify existence for a logarithmic number of trees in the history forest. We perform 100 appends per verification period.

5.3. Existing transparency logs

In this section, we compare the *empirical performance* of LegoLog and three specialized transparency logs. To set the goal for the empirical performance evaluation, we remind the reader that LegoLog’s contribution over existing transparency logs is that it is configurable as opposed to tailored for one specific setting, which is valuable for non-expert developers. As such, the purpose of the empirical evaluation is to show that the generality of LegoLog does not come at a substantial performance loss when compared to existing transparency logs, and not to beat the performance of other transparency logs (even though LegoLog is faster in a variety of settings). For each type of log, we report the throughput performance of the log server over multiple verification periods (t_v). We note that we



(a) Throughput for CONIKS and Merkle² workloads.



(b) Throughput for binary transparency.

Figure 13: LegoLog’s performance for three specialized logs.

report the results over singular runs, though we found the throughput behavior of the LegoLog server to be consistent over multiple runs of each experiment.

Merkle² [2]. Key transparency (KT) allows clients to check that a server correctly maintains a mapping of usernames to public keys. We described how to set developer parameters based on the workload of a hypothetical KT system for Signal in §2.3.1. We first compare LegoLog’s performance to Merkle² [2], which was designed for KT and runs on a log size of $N = 1M$. We scale the developer parameters (Table 2) back accordingly to match the log size of Merkle². For these parameters, LegoLog’s planner outputs $t_u = 15s$, $t_v = 15s$, $a = 2010$, `agghistory` = true, `verifier` = client. We plot the average throughput over time in Figure 13a. The blue circles mark when the history forest merges into one tree and consequently the lookup proof size decreases, which increases throughput for that period. LegoLog achieves an average throughput of 21,688 reads/s, compared to Merkle²’s 28,910 reads/s (LookUpPKVerify, averaged over 5 min). Our observed throughput measurements fell between 16,075 reads/s (p5) and 31,718 reads/s (p95), and this variation comes from how many trees are in the history forest at a given point in time. LegoLog achieves comparable performance to Merkle² for the workload Merkle² is designed for. LegoLog’s configurability comes with a small performance drop but offers a $\sim 5\times$ memory saving (25GB for Merkle² vs. 5GB for LegoLog).

CONIKS [1]. CONIKS runs on a log size of 10M, and we set the developer parameters (Table 2) to match their paper’s evaluation [1]. We compare against the version of CONIKS enhanced with the persistent data structure implemented in [2]. For these parameters, the planner outputs $t_u = 60m$, $t_v = 60m$, $a = 78210$, `agghistory` = false, `verifier` = client. We plot the average throughput over time in Figure 13a. CONIKS has an average throughput of 35,141 reads/s (averaged over 5 min), LegoLog has an average throughput of 39,005 reads/s. Our observed throughput measurements fell between 37,778 reads/s (p5) and 40,014 reads/s (p95).

Binary Transparency. Go’s checksum database enables users to authenticate their downloaded Go modules by maintaining the modules’ checksums on a transparency log [30]. We set the developer parameters (Table 2, row 2) based on Go’s deployed checksum database, which currently has 17M entries [31]. Every day $\sim 16k$ entries are added [32], averaging to 6 writes every 30s. A log size

TABLE 2: Example developer parameter settings for specialized transparency logs for different applications.

Specialized Log	N	t_W	burst_W	t_R	burst_R	t_{refresh}	t_{offline}	device_R	device_W	device_A
Merkle ² [2]	1M	15s	75	30s	2.4k	30m	24h	phone	phone	server
CONIKS [1]	10M	60m	75	30s	24k	60m	60m	phone	phone	server
Binary transparency (Go)	30M	30s	50	1s	500	24h	24h	laptop	laptop	laptop

of 30M can support 5 years worth of module checksums. Go has 2.7M users [33], and a conservative estimate of 10 module downloads/user/day yields an average of 313 reads/s (we can treat this as the baseline throughput that we wish LegoLog to achieve). The devices used by the reader, writer, and auditor are set to be laptops. LegoLog’s planner outputs $t_u = 30\text{s}$, $t_v = 55.5\text{m}$, $a = 7410$, $\text{agghistory} = \text{false}$, $\text{verifier} = \text{client}$. We plot the server’s request throughput over time in Figure 13b and show that LegoLog can achieve on average 5,725 reads/s. Our observed throughput measurements fell between 4,259 reads/s (p5) and 7,733 reads/s (p95). The fluctuation in throughput comes from the update periods that have elapsed within a verification period. When a verification period is over, the throughput suddenly increases because half of the update log has been cleared and merged into the verified base tree (circled in blue in Figure 13b).

5.4. API flexibility

We now evaluate *functionality* by showing that LegoLog can generate the functionality of 6 transparency logs via its API, and we include a comparison of asymptotic complexity. Table 3 shows how the LegoLog API can express a variety of applications. For each application, we show asymptotic costs based on the input developer parameters. Our goal is to show that when tailored to various applications, LegoLog overheads are comparable to existing work (i.e., configurability does not come at a significant performance cost).

In key transparency and certificate transparency (Row 1, Table 3), setting t_W can affect whether agghistory is turned on or off, and so can produce transparency logs with different asymptotic behavior. When agghistory is turned on, a lookup consists of looking up a value in the base tree of each history tree’s root ($O(\log^2 \frac{N}{a})$). When agghistory is turned off, a lookup just consists of looking up a value in the base tree of the last verification period ($O(\log \frac{N}{a})$). In either case, the client also needs to look up the proofs of inclusion or exclusion in the update trees ($O(e \log \frac{U}{ea})$). When agghistory is turned on, the auditor must verify that the history forest of base trees in each partition is append-only ($O(\log N)$). In either case, the auditor needs to check that the update log in each partition is append-only ($O(\log e + 1)$). When agghistory is turned on, monitoring consists of verifying an inclusion proof from a path of base trees in the history forest ($O(\log^2 \frac{N}{a})$). When agghistory is turned off, monitoring consists of verifying an inclusion proof from *each* verification period since the client last monitored the value ($O(E \log \frac{N}{a})$).

Certificate transparency [12], as deployed today, and binary transparency [15] (Row 2, Table 3) use chronological trees, which LegoLog can output by setting $t_v = \infty$ (i.e., the chronological update tree never gets rolled up into a base tree). Looking up and monitoring a value

consists of searching through all of the values in a shard ($O(\frac{N}{a})$). Auditing consists of an extension proof in each shard ($O(a \log U) \subseteq O(a \log N)$).

In the case of enhanced CT (Row 3, Table 3) and IoT authentication (Row 4, Table 3), LegoLog can produce comparable transparency logs when a powerful auditor device is specified as input (so that verifier is set to auditor) and when $t_v = t_u$. A lookup for a value consists of an inclusion proof in a partition ($O(\log \frac{N}{a})$) and inclusion/exclusion proofs in the partition’s update trees ($O(e \log \frac{U}{ea})$). Auditing consists of validating that the update log in each partition is append-only ($O(a(\log e + 1))$) and that update log entries have been correctly rolled into the next base tree ($O(N)$). The client only needs to keep track of the digest when monitoring ($O(1)$). In encrypted backups [7] (Row 5, Table 3), the threat model differs slightly from LegoLog’s because clients trust a subset of the hardware security modules, enabling SafetyPin to achieve asymptotically better auditing costs.

6. Related Work

Transparency logs. A rich line of work has examined improving transparency logs. CONIKS was the first academic work to propose a transparency log for public keys [1] and backs Apple’s deployment of iMessage key transparency [5]. It relies on the owner to monitor their key every epoch, which can be impractical for short epochs. LegoLog enables trading off less frequent monitoring for more expensive lookups. ECT [37] and WAVE [6] maintain both a chronological tree and a prefix tree, but require an auditor to check the correspondence between the two, which is expensive. Google’s Trillian [35], [38] is mainly used for certificate transparency, and it provides a verifiable log abstraction.

Merkle² maintains a prefix tree inside each node of a chronological tree [2], enabling efficient monitoring proofs. Merkle² clients that go offline for a while can discover that a server misbehaved but cannot prevent this misbehavior. LegoLog enables the client to prevent misbehavior by trading off performance for longer verification periods.

OPTIKS [39], Parakeet [3], and SEEMless [22] build off of CONIKS [1] and use Merkle Patricia Tries (MPTs) as their core data structure. Ghosh and Chase describe an auditor-free version of OPTIKS [40]. Parakeet reduces storage space in comparison to SEEMless by storing the epoch in the leaf node, allowing for any epoch’s MPT to be reconstructed. Similarly, LegoLog also allows for any verification period’s tree to be efficiently reconstructed (§5.1).

Keybase was the first deployment of a publicly auditable public key directory [41]. WhatsApp recently deployed a key transparency system [4] based on SEEMless [22] and Parakeet [3]. Cloudflare’s validation service

TABLE 3: Asymptotics of lookup, auditing, and monitoring work for transparency logs derived by LegoLog for applications, compared to existing work. N is the number of entries in the log, U is the number of updates per verification period, a is the number of partitions, E is the number of verification periods since the value was last monitored, and e is the number of update periods in a verification period.

Application	Existing work				LegoLog		
	Work	Lookup	Audit	Monitor	Lookup	Audit	Monitor
Key transparency (KT) [1] KT for IOT [34]	[2]	$O(\log^2 N)$	$O(\log N)$	$O(\log^2 N)$	$O(\log^2 \frac{N}{a} + e \log \frac{U}{ea})$	$O(a(\log N + \log e + 1))$	$O(\log^2 \frac{N}{a})$
Certificate transparency (CT) [12] CT for IOT	[1]	$O(\log N)$	$O(1)$	$O(E \log N)$	$O(\log \frac{N}{a} + e \log \frac{U}{ea})$	$O(a(\log e + 1))$	$O(E \log \frac{N}{a})$
CT—deployed today Binary transparency [30], [36]	[35]	$O(N)$	$O(\log N)$	$O(N)$	$O(\frac{N}{a})$	$O(a \log N)$	$O(\frac{N}{a})$
Enhanced CT	[37]	$O(\log N)$	$O(N)$ or $O(\log N)$	$O(1)$ or $O(N)$	$O(\log \frac{N}{a} + e \log \frac{U}{ea})$	$O(a(\log e + 1) + N)$	$O(1)$
IOT authentication	[6]	$O(\log N)$	$O(N)$	$O(1)$	$O(\log \frac{N}{a} + e \log \frac{U}{ea})$	$O(a(\log e + 1) + N)$	$O(1)$
Encrypted backups	[7]	$O(\log N)$	$O(\log N)$	$O(1)$	$O(\log \frac{N}{a} + e \log \frac{U}{ea})$	$O(a(\log e + 1) + N)$	$O(1)$

audits the WhatsApp deployment [42], and an IETF working group is examining how to make transparency systems simple to run across organizations [43]. Elektra [44] builds off of SEEMless and targets the use case where users have multiple devices, using a form of signature chains similar to how users in LegoLog and Merkle² [2] sign updates (§3.4).

Binary transparency [8], [9], [10] builds off of certificate transparency [12] and is used for validating software updates. LegoLog can derive transparency logs for such workloads. Recent work also showed how to build a verifiable registry for voter registration that is backwards-compatible with existing systems [45].

Byzantine fault tolerance. In the decentralized system model, transparency logs can be implemented using consensus protocols [46], [47], [48], [49], [50], [51], [52], [53], [54], [55], [56]. LegoLog focuses on the single-server model and does not require heavyweight consensus protocols. Prior work also uses a public blockchain to publish an auditable directory, an approach constrained by the performance of the underlying blockchain. EthIKS [57] and Catena [58] show how to use the public blockchain as a public ledger to commit to a CONIKS directory and minimize client bandwidth for auditing.

Heavyweight cryptographic primitives. Another approach to transparency logs is to use powerful, but more expensive, cryptographic primitives such as succinct non-interactive arguments of knowledge (SNARKs) [17], [18], [19], accumulators [20] or vector commitments [21] that are more expensive to compute, but have better asymptotic performance. LegoLog does not capture transparency logs based on these cryptographic primitives. While these logs improve asymptotics, they tend to have high constants and so high overheads in practice.

7. Discussion

Scalability considerations. LegoLog’s scalability is primarily determined by server and auditor resources. Our planner automatically calculates and outputs the minimum server requirements (CPU, bandwidth, and storage) needed for a specific deployment’s performance goals. The system scales horizontally by increasing the number of partitions (a), which divides the log into independent shards. This partitioning allows lookups and verification operations

to be parallelized across shards, significantly improving throughput for large deployments. As shown in our evaluation, increasing the number of partitions reduces both client and server work per operation, enabling LegoLog to scale to logs with millions of entries (§5.3). The trade-off is increased auditor work, which grows linearly with the number of partitions, and so this parameter should be optimized based on the specific workload constraints and available resources.

8. Conclusion

LegoLog is a configurable transparency log that can *automatically* output a suitable log design given a workload description. It enables non-experts to use transparency logs, thus enabling more applications to leverage them. We show that LegoLog provides configurability while performing comparably to special-purpose transparency logs.

Acknowledgment

We are grateful to our anonymous shepherd for their feedback, as well as to the anonymous EuroS&P reviewers for their suggestions, which improved the presentation of this paper. We thank Yuncong Hu for guidance with the Merkle² implementation, and the Sky security students for their helpful feedback. We also thank Henry Corrigan-Gibbs for his support. This work is supported in part by NSF Award CNS-2054869 and gifts from Accenture, AMD, Anyscale, Apple, Capital One, Cisco, Facebook, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Microsoft, Mozilla, NVIDIA, Samsung SDS, SAP, and VMware. This work is also supported by an NSF Graduate Research Fellowship and a Microsoft Ada Lovelace Research Fellowship.

References

- [1] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, “CONIKS: Bringing key transparency to end users,” in *USENIX Security*, 2015.
- [2] Y. Hu, K. Hooshmand, H. Kalidhindi, S. J. Yang, and R. A. Popa, “Merkle²: A low-latency transparency log system,” in *IEEE S&P*, 2021.
- [3] H. Malvai, L. Kokoris-Kogias, A. Sonnino, E. Ghosh, E. Oztürk, K. Lewi, and S. Lawlor, “Parakeet: Practical key transparency for end-to-end encrypted messaging,” 2023, <https://eprint.iacr.org/2023/081>. [Online]. Available: <https://eprint.iacr.org/2023/081>

- [4] S. Lawlor and K. Lewi, "Deploying key transparency at WhatsApp," <https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/>.
- [5] Apple, "Advancing iMessage security: iMessage contact key verification," <https://security.apple.com/blog/imessage-contact-key-verification/>.
- [6] M. P. Andersen, S. Kumar, M. AbdelBaky, G. Fierro, J. Kolb, H.-S. Kim, D. E. Culler, and R. A. Popa, "WAVE: A decentralized authorization framework with transitive delegation," in *USENIX Security*, 2019.
- [7] E. Dauterman, H. Corrigan-Gibbs, and D. Mazières, "SafetyPin: Encrypted backups with human-memorable secrets," in *OSDI*, 2020.
- [8] B. Hof and G. Carle, "Software distribution transparency and auditability," *arXiv preprint arXiv:1711.07278*, 2017.
- [9] K. Nikitin, E. Kokoris-Kogias, P. Jovanovic, N. Gailly, L. Gasser, I. Khoffi, J. Capps, and B. Ford, "CHAINIAC: Proactive Software-Update transparency via collectively signed skipchains and verified builds," in *USENIX Security*, 2017.
- [10] M. Al-Bassam and S. Meiklejohn, "Contour: A practical system for binary transparency," in *DPM/CBT@ESORICS*, 2018.
- [11] "Model transparency," <https://github.com/google/model-transparency>.
- [12] B. Laurie, A. Langley, and E. Kasper, "Certificate transparency," rFC 6962. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6962>
- [13] K. Nissim and M. Naor, "Certificate revocation and certificate update," in *USENIX Security*, 1998.
- [14] R. Tamassia, "Authenticated data structures," in *ESA*, 2003.
- [15] Binary Transparency, "Building trust in the software supply chain," <https://binary.transparency.dev>.
- [16] S. Lawlor and K. Lewi, "Deploying key transparency at whatsapp," <https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/>. [Online]. Available: <https://engineering.fb.com/2023/04/13/security/whatsapp-key-transparency/>
- [17] W. Chen, A. Chiesa, E. Dauterman, and N. P. Ward, "Reducing participation costs via incremental verification for ledger systems," *Cryptology ePrint Archive*, 2020.
- [18] I. Tzialla, A. Kothapalli, B. Parno, and S. Setty, "Transparency dictionaries with succinct proofs of correct operation," in *NDSS*, 2022.
- [19] N. Tyagi, B. Fisch, A. Zitek, J. Bonneau, and S. Tessaro, "VeRSA: Verifiable registries with efficient client audits from RSA authenticated dictionaries," in *CCS*, 2022.
- [20] A. Tomescu, V. Bhupatiraju, D. Papadopoulos, C. Papamanthou, N. Triandopoulos, and S. Devadas, "Transparency logs via append-only authenticated dictionaries," in *CCS*, 2019.
- [21] D. Leung, Y. Gilad, S. Gorbunov, L. Reyzin, and N. Zeldovich, "Aardvark: A concurrent authenticated dictionary with short proofs," *IACR Cryptol. ePrint Arch.*, vol. 2020, p. 975, 2020.
- [22] M. Chase, A. Deshpande, E. Ghosh, and H. Malvai, "SEEMless: Secure end-to-end encrypted messaging with less trust," in *ACM CCS*, 2019.
- [23] "Key transparency new design," github.com/google/keytransparency/blob/master/docs/design_new.md.
- [24] "The battle inside Signal," <https://www.theverge.com/22249391/signal-app-abuse-messaging-employees-violence-misinformation>.
- [25] "Signal revenue & usage statistics," <https://www.businessofapps.com/data/signal-statistics/>.
- [26] Signal, "The difficulty of private contact discovery," 2014, <https://signal.org/blog/contact-discovery/>.
- [27] R. Merkle, "Method of providing digital signatures," Patent US4 309 569A, 1979. [Online]. Available: <https://patents.google.com/patent/US4309569>
- [28] Apple, "Energy efficiency guide for iOS apps," <https://developer.apple.com/library/archive/documentation/Performance/Conceptual/EnergyGuide-iOS/index.html>.
- [29] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, "Making data structures persistent," in *ACM STOC*, 1986.
- [30] K. Hockman, "Module mirror and checksum database launched," <https://go.dev/blog/module-mirror-launch>.
- [31] Golang, "go.sum database tree," <https://sum.golang.org/latest>.
- [32] —, "index.golang.org," <https://index.golang.org/index?since=2022-03-03T19:08:52.997264Z>.
- [33] E. Zharova, "The state of Go," <https://blog.jetbrains.com/go/2021/02/03/the-state-of-go/>.
- [34] H. Shafagh, L. Burkharter, S. Ratnasamy, and A. Hithnawi, "Droplet: Decentralized authorization and access control for encrypted data streams," in *USENIX Security*, 2020.
- [35] Google, "Trillian tessera," <https://github.com/transparency-dev/trillian-tessera>.
- [36] A. Delignat-Lavaud, C. Fournet, K. Vaswani, S. Clebsch, M. Riechert, M. Costa, and M. Russinovich, "Why should I trust your code? Confidential computing enables users to authenticate code running in TEEs, but users also need evidence this code is trustworthy," *ACM Queue*, 2023.
- [37] M. D. Ryan, "Enhanced certificate transparency and end-to-end encrypted mail," in *NDSS*, 2014.
- [38] Google, "Trillian," <https://github.com/google/trillian>.
- [39] J. Len, M. Chase, E. Ghosh, K. Laine, and R. C. Moreno, "OPTIKS: An optimized key transparency system," *Cryptology ePrint Archive*, Paper 2023/1515, 2023, <https://eprint.iacr.org/2023/1515>. [Online]. Available: <https://eprint.iacr.org/2023/1515>
- [40] E. Ghosh and M. Chase, "Weak consistency mode in key transparency: Optiks," *Cryptology ePrint Archive*, 2024.
- [41] K. Ryan, T. Pornin, and S. Fitzgerald, "Protocol security review – keybase," https://keybase.io/docs-assets/blog/NCC_Group_Keybase_KB2018_Public_Report_2019-02-27_v1.3.pdf.
- [42] T. Meunier and M. Galicer, "Cloudflare helps verify the security of end-to-end encrypted messages by auditing key transparency for WhatsApp," <https://blog.cloudflare.com/key-transparency/>.
- [43] IETF, "Key transparency (keytrans)," <https://datatracker.ietf.org/wg/keytrans/about/>.
- [44] J. Len, M. Chase, E. Ghosh, D. Jost, B. Kesavan, and A. Marcedone, "ELEKTRA: Efficient lightweight multi-device key transparency," in *CCS*, 2023.
- [45] A. Fábrega, J. Cable, M. A. Specter, and S. Park, "Cryptographic verifiability for voter registration systems," *arXiv preprint arXiv:2503.03974*, 2025.
- [46] M. Baudet, A. Ching, A. Chursin, G. Danezis, F. Garillot, Z. Li, D. Malkhi, O. Naor, D. Perelman, and A. Sonnino, "State machine replication in the Libra blockchain," *The Libra Assn., Tech. Rep.*, 2019.
- [47] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich *et al.*, "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *EuroSys*, 2018.
- [48] M. Castro and B. Liskov, "Practical Byzantine fault tolerance," in *OSDI*, 1999.
- [49] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "Hotstuff: BFT consensus with linearity and responsiveness," in *ACM PODC*, 2019.
- [50] S. Duan, S. Peisert, and K. N. Levitt, "hBFT: speculative Byzantine fault tolerance with minimum cost," *IEEE Transactions on Dependable and Secure Computing*, 2014.
- [51] R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, "The next 700 BFT protocols," in *EuroSys*, 2010.
- [52] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, "SBFT: a scalable and decentralized trust infrastructure," in *IEEE DSN*, 2019.
- [53] R. Kotla, L. Alvisi, M. Dahlin, A. Clement, and E. Wong, "Zyzyva: Speculative Byzantine fault tolerance," *ACM TOCS*, 2010.
- [54] D. Malkhi, K. Nayak, and L. Ren, "Flexible Byzantine fault tolerance," in *SIGSAC*, 2019.
- [55] L. Lamport, "Byzantizing Paxos by refinement," in *International symposium on distributed computing*. Springer, 2011.

- [56] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized business review*, 2008.
- [57] J. Bonneau, "EthIKS: Using Ethereum to audit a CONIKS key transparency log," in *Financial Cryptography*, 2016.
- [58] A. Tomescu and S. Devadas, "Catena: Efficient non-equivocation via Bitcoin," in *IEEE S&P*, 2017.

Appendix A.

Statement on data availability

The artifacts necessary to reproduce the results presented in this work, including reference datasets, code, and any additional documentation, are open-sourced at this link: <https://github.com/vivi/legolog>.

Appendix B.

Protocol specification

We now fully specify the implementation of the LegoLog protocol, implementing the API specified in §2.2 of the main paper.

Building blocks

We first define the syntax for existing building blocks.

Chronological Merkle Tree

$\text{ChronTree.Append}(\text{val})$: Appends val to the tree.

$\text{ChronTree.ProveExtension}(\text{root}_{\text{old}}, \text{root}) \rightarrow \pi$: Generate proof that the tree rooted at root is an extension of the tree rooted at root_{old} . A detailed algorithm for generating the extension proof is found in §2.1.4.1. of [12].

$\text{ChronTree.VerifyExtension}(\pi, \text{root}_{\text{old}}, \text{root}) \rightarrow \{0, 1\}$: Outputs 1 if the extension proof π is valid and 0 otherwise.

$\text{ChronTree.GetPath}(i) \rightarrow (\text{val}, \text{Path})$: Outputs the value and Merkle path associated with i -th inserted value.

$\text{ChronTree.CheckPath}(\text{root}, i, \text{val}, \text{Path}) \rightarrow \{0, 1\}$: Outputs 1 if the result of hashing Path , i , and val matches root and 0 otherwise.

$\text{ChronTree}[i]^* \rightarrow (\text{val}, \{0, 1\})$: Outputs the i -th inserted val and whether it has a valid proof by calling $\text{ChronTree.CheckPath}(\text{root}, i, \text{ChronTree.GetPath}(i))$.

$\text{ChronTree}[i] \rightarrow \text{val}$: Outputs the i -th inserted val .

Chronological Merkle Forest

$\text{ChronForest.Append}(\text{val})$: Appends val to the forest.

$\text{ChronForest.ProveExtension}(\text{roots}_{\text{old}}, \text{roots}) \rightarrow \pi$: Generate proof that the forest with roots roots is an extension of the forest with roots $\text{roots}_{\text{old}}$. A detailed algorithm for generating the extension proof for chronological trees is found in §2.1.4.1. of [12] and a detailed algorithm for generating the extension proof for chronological forests is found in §V.A of [2].

$\text{ChronForest.VerifyExtension}(\pi, \text{roots}_{\text{old}}, \text{roots}) \rightarrow \{0, 1\}$: Outputs 1 if the extension proof π is valid and 0 otherwise.

$\text{ChronForest.GetPath}(i) \rightarrow (\text{val}, \text{Path})$: Outputs the value and Merkle path associated with i -th inserted value.

$\text{ChronForest.CheckPath}(\text{root}, i, \text{val}, \text{Path}) \rightarrow \{0, 1\}$: Outputs 1 if the result of hashing Path , i , and val matches root and 0 otherwise.

$\text{ChronForest}[i]^* \rightarrow (\text{val}, \{0, 1\})$: Outputs the i -th inserted val and whether it has a valid proof by calling $\text{ChronForest.CheckPath}(\text{root}, i, \text{ChronForest.GetPath}(i))$.

$\text{ChronForest}[i] \rightarrow \text{val}$: Outputs the i -th inserted val .

Prefix Merkle Tree

$\text{PrefixTree.BuildPrefixTree}(\text{idvals}) \rightarrow \text{PrefixTree}$:

Constructs a Merkle prefix tree given id -value pairs idvals .

$\text{PrefixTree.GetPath}(\text{id}) \rightarrow (\text{val}, \text{Path})$: Outputs the value and Merkle path associated with id in a Merkle prefix tree. If id is in the tree, Path consists of the siblings of the nodes in the path from id to the tree's root. If id is not in the tree, $\text{val} \leftarrow \perp$ and Path is a proof of exclusion which is the longest-prefix match of id currently in the tree (either a leaf node $\text{id}' \neq \text{id}$ that matches the first few bits of id or an empty node that is a prefix of id).

$\text{PrefixTree.CheckPath}(\text{root}, \text{id}, \text{val}, \text{Path}) \rightarrow \{0, 1\}$: Outputs 1 if the result of hashing Path , id , and val matches root and 0 otherwise.

$\text{PrefixTree.Insert}(\text{id}, \text{val})$: Inserts id , val into the tree.

Signatures

$\text{Sign}(\text{sk}, \text{val}) \rightarrow \sigma$: Signs val with signing key sk .

$\text{VerifySig}(\text{vk}, (\text{val}, \sigma)) \rightarrow \{0, 1\}$: Verifies signature σ .

Server

$\text{config} := (t_u, t_v, a, \text{agghistory}, \text{verifier})$, as described in Table 2 in the main paper.

$\text{Server}[\alpha] := (\text{QueryBT}, \text{VerifyBT}, \text{UpdateLog}, \text{HistoryForest}, \text{CurrUpdates}, t, \text{HashChain})$

- α is the partition index.
- QueryBT is a PrefixTree .
- VerifyBT is a PrefixTree .
- UpdateLog is a ChronTree of PrefixTrees from each update period.
- HistoryForest is a ChronForest consisting of base trees from each verification period.
- CurrUpdates is a list of updates in the current update period.
- t is the current epoch.
- HashChain is a hash chain of the server's state.

Notation: $\text{Server}[\alpha][\text{id}]$ denotes the value of id in partition α .

$\text{Server}[\alpha].\text{Write}(\text{id}, (\text{val}, \sigma))$

1: $\text{Server}[\alpha].\text{CurrUpdates} \leftarrow ((\text{val}, \sigma), \text{Server}[\alpha].\text{CurrUpdates})$

$\text{Server.NextEpoch}(\text{config})$

1: **for** $\alpha \leftarrow 1, \dots, \text{config}.a$ **do**
2: $\text{Server}[\alpha].\text{NextEpoch}(\text{config})$
3: **end for**

$\text{Server}[\alpha].\text{NextEpoch}(\text{config})$

1: $\text{updateTree} \leftarrow \text{BuildPrefixTree}(\text{Server}[\alpha].\text{CurrUpdates})$
2: $\text{Server}[\alpha].\text{UpdateLog.Append}(\text{updateTree})$
3: **if** $\text{Server}[\alpha].t + 1 \bmod \text{config}.t_v = 0$ **then**
4: **if** $\text{config}.agghistory$ **then**
5: $\text{Server}[\alpha].\text{HistoryForest.Append}(\text{Server}[\alpha].\text{QueryBT})$


```

6:   else
7:     Server[α].HashChain ← H(
      Server[α].QueryBT.root,
      Server[α].VerifyBT.root,
      Server[α].t, Server[α].HashChain)
8:   end if
9:   Server[α].QueryBT ← Server[α].VerifyBT
10:  for ut ∈ Server[α].UpdateLog do
11:    for (id, val) ∈ ut do
12:      v ← Server[α][id]
13:      if v = ⊥ then
14:        v.vk ← val
15:      end if
16:      v.vals ← (val, v.vals)
17:      v.last ← val
18:      Server[α].VerifyBT.Insert(id, v)
19:    end for
20:  end for
21:  Server[α].UpdateLog ← ()
22: end if
23: Server[α].CurrUpdates ← ()
24: Server[α].t ← Server[α].t + 1

Server.GetDigest() → dgst
1: dgst ← ()
2: for α ← 1, ..., config.a do
3:   dgst[α] ← Server[α].GetDigest()
4: end for
5: Output dgst

Server[α].GetDigest() → dgst[α]
1: Output (t, Server[α].QueryBT.root,
      Server[α].VerifyBT.root,
      Server[α].HistoryForest.roots,
      Server[α].UpdateLog.root, Server[α].HashChain)

Server.Prove(config, told, tnew) → π
1: π ← ()
2: for α ← 1, ..., config.a do
3:   π[α] ← Server[α].Prove(config, told, tnew)
4: end for
5: Output π

Server[α].Prove(config, told, tnew) → π[α]
1: Server[α]t := the state of the server's partition α at
   epoch t.
2: if config.agghistory then
3:   π[α].hf ← ChronForest.ProveExtension(
      Server[α]told.HistoryForest.roots,
      Server[α]tnew.HistoryForest.roots)
4: else
5:   π[α].hc ← ((Server[α]told+1.QueryBT.root,
      Server[α]told+1.VerifyBT.root, told + 1),
      ...,
      (Server[α]tnew-1.QueryBT.root,
      Server[α]tnew-1.VerifyBT.root, tnew - 1))
6: end if
7: if ⌊told/config.tv⌋ = ⌊tnew/config.tv⌋ then
8:   π[α].ul ← ChronTree.ProveExtension(
      Server[α]told.UpdateLog.root,
      Server[α]tnew.UpdateLog.root)
9: end if
10: Output π

```

Auditor

$dgst[α] := (t, qroot, vroot, hfroots, ulroot, HashChain)$

- t is the current epoch.
- $qroot$ is the root of *QueryBT*.
- $vroot$ is the root of *VerifyBT*.
- $hfroots$ are the roots of the *HistoryForest*.
- $ulroot$ is the root of *UpdateLog*.
- $HashChain$ is the hash chain of the server's state.

Auditor.GetDigest(t) → dgst:

Auditor.Audit(config, dgst_{old}, dgst_{new}, π) → {0, 1}:

```

1: ok ← 1
2: for α ← 1, ..., config.a do
3:   if config.agghistory then
4:     ok ← ok ∧ VerifyExtension(π[α].hf,
      dgstold[α].hfroots, dgstnew[α].hfroots)
5:   else
6:     h' ← dgstold[α].HashChain
7:     for n ∈ π[α].hc do
8:       h' ← H(n, h')
9:     end for
10:    ok ← ok ∧ h' = dgstnew[α].HashChain
11:  end if
12:  if dgstold[α].t mod config.tv =
      dgstnew[α].t mod config.tv then
13:    ok ← ok ∧ VerifyExtension(π[α].ul,
      dgstold[α].ulroot, dgstnew[α].ulroot)
14:  end if
15: end for
16: Output ok

```

Client

config := ($t_u, t_v, a, agghistory, verifier$), as described in Table 2 in the main paper.

Client := (*LastMonitored*, *VBTRoots*)

- *LastMonitored* is a map of identifiers id to the latest verification period the client has monitored for that value.
- *VBTRoots* is a map of identifiers id to a set of base tree roots that have already been verified by the client (for that id).

Client.Read(config, dgst, id, t) → val

```

1: ok ← 1
2: α ← H(id) mod config.a
3: root ← dgst[α].qroot
4: vals, Pathid ← Server[α].QueryBT.GetPath(id)
5: ok ← ok ∧ CheckPath(root, id, vals, Pathid)
6: if config.agghistory then
7:   for rooth ∈ Server[α].HistoryForest.roots do
8:     th ← verification period rooth corresponds to
9:     valh, Pathid,h ←
      Server[α].HistoryForest[th].GetPath(id)
10:    ok ← ok ∧ CheckPath(rooth, id, valh, Pathid,h)
11:  ok ← ok ∧ valh.vals ⊆ val.vals
12: end for
13: h ← 0
14: r ← ⌊t/tv⌋

```

```

15: end if
16: if VerifySig(vals.vk, vals.last) then
17:   vallatest ← vals.last
18: end if
19: for  $u = 0, \dots, \lfloor (t - t_v \cdot \lfloor t/t_v \rfloor) / t_u \rfloor$  do
20:   (treeu, oku) ← dgst[α].UpdateLog[u]*
21:   ok ← ok ∧ oku
22:   rootu ← treeu.root
23:   valu, Pathid,u ← treeu.GetPath(id)
24:   ok ← ok ∧ CheckPath(rootu, id, valu, Pathid,u)
25:   if valu ≠ ⊥ ∧ VerifySig(vk, valu) = 1 then
26:     vallatest ← valu
27:   end if
28: end for
29: if ok then
30:   Output vallatest
31: else
32:   Output ⊥
33: end if

```

Client.Write(config, sk, id, val, t) → {0, 1}

```

1: σ ← Sign(sk, (id, val))
2: α ← H(id) mod config.a
3: Server[α].Write(id, (val, σ))
4: Wait until epoch t + 1
5: dgst ← Auditor.GetDigest(t + 1)
6: vallatest ← Client.Read(config, dgst, id, t + 1)
7: Output vallatest  $\stackrel{?}{=}$  val

```

Client.Monitor(config, dgst, id, val, t) → {0, 1}

```

1: ok ← 1
2: tm ← ⌊t/config.tv⌋
3: α ← H(id) mod config.a
4: val', Path ← Server[α].VerifyBT.GetPath(id)
5: ok ← ok ∧ CheckPath(dgst[α].vroot, id, val', Path)
6: ok ← ok ∧ val'  $\stackrel{?}{=}$  val
7: if config.agghistory then
8:   t' ← verification period id was updated to val
9:   for tl ∈ path from t' to root in
       Server[α].HistoryForest do
10:    dgstl ← Auditor.GetDigest(tl)
11:    val', Path ← Server[α].HistoryForest[tl].
        GetPath(id)
12:    ok ← ok ∧ CheckPath(dgstl[α].vroot,
        id, val', Path)
13:    ok ← ok ∧ val'  $\stackrel{?}{=}$  val
14:   end for
15: else
16:   for tl ← Client.LastMonitored[id], ..., tm do
17:    dgstl ← Auditor.GetDigest(tl)
18:    val', Path ← Server[α].HistoryForest[tl].
        GetPath(id)
19:    ok ← ok ∧ CheckPath(dgstl[α].vroot,
        id, val', Path)
20:    ok ← ok ∧ val'  $\stackrel{?}{=}$  val
21:   end for
22: end if
23: Client.LastMonitored[id] ← tm
24: Output ok

```

Appendix C. Security analysis

Building blocks

Before we prove the security of LegoLog, we first introduce security definitions for Merkle trees that are used as building blocks in LegoLog's core data structure. These definitions and security proofs for Merkle trees have already been established [27], [2], [12], but we review them here before moving on to prove the security of LegoLog itself.

Security of prefix Merkle trees

Prefix Merkle trees, as introduced in §3.1 of the main paper, have their leaves organized in lexicographic order, and internal nodes store the range of the child nodes.

Lemma 3. *Let T be a prefix Merkle tree containing a leaf node L with identifier id and value val . Let h be the hash function used to construct T . If an adversary is able to generate a proof π for the leaf (id, val') where $val' \neq val$ that verifies against the root hash R of T , then the adversary has found a collision in the hash function h .*

Proof. Since T contains the leaf (id, val) , there must exist a path P from L to R consisting of nodes $n_1, n_2, \dots, n_k$ such that:

$$\begin{aligned}
 n_1 &= h(L) \\
 n_2 &= h(n_1, n'_1) \text{ for some } n'_1 \\
 &\vdots \\
 n_k &= R
 \end{aligned}$$

Similarly, π defines a path P' consisting of nodes $n'_1, n'_2, \dots, n'_k$ such that:

$$\begin{aligned}
 n'_1 &= h(L') \text{ where } L' = (id, val') \\
 n'_2 &= h(n'_1, n''_1) \text{ for some } n''_1 \\
 &\vdots \\
 n'_k &= R
 \end{aligned}$$

Since $val \neq val'$, we have $L \neq L'$. Therefore, $n_1 = h(L) \neq h(L') = n'_1$. However, both paths P and P' lead to the same root hash R . Therefore, there exist distinct values n_1 and n'_1 that hash to the same value $n_2 = n'_2$. This means h has a collision. $\square$

Security of chronological Merkle trees

Chronological Merkle trees, as introduced in §3.1 of the main paper, have leaves organized by time the leaf was appended to the tree.

Lemma 4. *Let T be a chronological Merkle tree. Let h be the hash function used to construct T . Suppose T contains leaves $L_1, L_2, \dots, L_n$ that were appended in that order. Let $L_n = (val)$. Let R be the root hash of T . If an adversary is able to generate a proof π that a leaf $L'_n = (val')$ was appended to T after L_{n-1} , where $val' \neq val$, then the adversary has found a collision in the hash function h .*

Proof. The adversary generates a proof π that T contains L'_n under the same root R . As proved in Lemma 3, this

is possible only if the adversary has found a collision in the hash function h . $\square$

Lemma 5. *Let T_n be a chronological Merkle tree containing leaves $L_1, \dots, L_n$ appended in order. Let $\pi_{j \rightarrow n}$ be an extension proof showing T_n extends T_j , for $j < n$. If an adversary generates an extension proof $\pi'_{j \rightarrow n}$ showing tree T'_n extends T_j , where T'_n contains a leaf $L'_i = (\text{val}'_i)$ such that $i < j$ and $\text{val}'_i \neq \text{val}_i$, then the adversary has found a collision in h .*

Proof. Let R_j be the root of T_j and R'_n be the root of T'_n . Since π' shows T'_n extends T_j , $\pi'_{j \rightarrow n}$ must contain the hashes needed to compute R_j , and the hashes needed to compute R'_n . Since T_j contains the leaf $L_i = (\text{val}_i)$, there must exist a path P from L to R_j consisting of nodes $n_1, n_2, \dots, n_k$ such that:

$$\begin{aligned} n_1 &= h(L_i) \\ n_2 &= h(n_1, n'_1) \text{ for some } n'_1 \\ &\vdots \\ n_k &= R_j \end{aligned}$$

Similarly, if T'_n contains a leaf $L'_i = (\text{val}'_i)$ such that $i < j$ and $\text{val}'_i \neq \text{val}_i$, then $\pi'_{j \rightarrow n}$ must define a path P' from L'_i to R_j consisting of nodes $n'_1, n'_2, \dots, n'_k$ such that:

$$\begin{aligned} n'_1 &= h(L'_i) \\ n'_2 &= h(n'_1, n''_1) \text{ for some } n''_1 \\ &\vdots \\ n'_k &= R_j \end{aligned}$$

Since $\text{val}_i \neq \text{val}'_i$, $n_1 = h(L) \neq h(L') = n'_1$. However, both paths P and P' lead to the same root hash R_j . Therefore, there exist distinct values n_1 and n'_1 that hash to the same value $n_2 = n'_2$. Thus, the adversary has found a collision in h . $\square$

Security game

We define security for LegoLog via a security game (Figure 14, Experiment 6). In our security game, we model the case where honest clients interact with a server controlled by the adversary. In our model, all honest clients also interact with at least one honest auditor and all have the same view of the server state. To capture this in the security game, the challenger plays the role of the honest clients and an auditor. The adversary sends one digest per round to the challenger, capturing the fact that all honest clients have the same view. The adversary can choose the identifier-value pairs that the honest clients write and, for each round, choose which identifier-value pairs are monitored and which are not. Before the game ends, the challenger audits and reads all identifier-value pairs. The adversary's goal is to pass all auditing and monitoring checks while ensuring that a read outputs a value that does not match the latest write from an honest client.

This security game captures both detection and prevention of log tampering. For detection, the adversary can choose for each round, which identifier-value pairs are monitored and which are not, and the challenger must

detect any tampering in the last monitoring check on all identifier-value pairs inserted by the challenger. For prevention, in the last round, the challenger reads without doing any monitoring, and so any misbehavior must be caught by the auditing proof or the monitoring proof in the previous round(s).

Security proof

We now prove Theorem 1 from the main paper. At a high level, we must reduce the advantage of the adversary in Experiment 6 to the advantage of an adversary against the signature scheme or the collision-resistant hash function. We can prove this via a hybrid argument.

***Hybrid game.** We construct a hybrid security game (Figure 15, Experiment 7) where we modify the win conditions of the original security game so that the adversary only wins if the challenger outputs an old (overwritten) value for a given identifier as a result of a read. The difference between the original game and this hybrid game is precisely the probability that the adversary can convince the challenger to accept a new value that was never written with the original secret key associated with that identifier.

In Figure 16, Experiment 8, we define $\mathcal{A}_\Delta$ to be the adversary that can win the difference in the games of Experiment 6 and Experiment 7. That is, $\mathcal{A}_\Delta$'s goal is to pass all auditing and monitoring checks while ensuring that read outputs a value that does not match *any* previous write from an honest client.

In Figure 17, we use the adversary $\mathcal{A}_\Delta$ to construct an adversary $\mathcal{B}$ in the EUF-CMA security game, where $\mathcal{B}$ plays the challenger in our security game and also interfaces between the challenger in the EUF-CMA game and the adversary $\mathcal{A}_\Delta$, sending identifier-value pairs from the adversary $\mathcal{A}_\Delta$ to the EUF-CMA challenger as messages and using signature responses from the EUF-CMA challenger to construct messages to send to the adversary $\mathcal{A}$ as a part of write requests. $\mathcal{B}$ fixes the signature scheme for one of the identifiers id_{pk} that $\mathcal{A}_\Delta$ sends to be interacting with the challenger in the EUF-CMA game. For other identifiers, $\mathcal{B}$ samples a new signing key (if it does not yet exist) and uses that when writing values for that identifier to the server. If $\mathcal{A}_\Delta$ wins, then it has produced a new value that was never written (with a valid signature) with the original key associated with that identifier. If this identifier is id_{pk} , then $\mathcal{B}$ can send this back to the EUF-CMA challenger to win the game. Since $\mathcal{A}_\Delta$ cannot distinguish between which identifier is using the EUF-CMA challenger and is using $\mathcal{B}$'s sampled signing keys, $\mathcal{B}$ will succeed with probability $1/\text{poly}(\cdot)$. Therefore, the difference between the adversary's advantage in the original security game and in the first hybrid game is simply the advantage an EUF-CMA adversary (with some polynomial factors) which, given a secure signature scheme, is negligible.

We now prove that if the adversary wins in this modified game (Experiment 7), then we can construct an adversary $\mathcal{C}$ that can find a collision in the hash function. We prove this for the case when $\text{agghistory} = \text{false}$ and $\text{agghistory} = \text{true}$ separately.

Case 1. We first argue security when $\text{agghistory} = \text{false}$. Assume that the challenger reads an old value val' for id when the last write to id was val ($\text{agghistory} =$

Experiment 6. The below security game for a configurable transparency log against an adversary $\mathcal{A}$ is parameterized by a configuration config , which contains the security parameter λ (see Table 2 in the main paper).

The game proceeds as follows:

- The challenger initializes dictionary dict and round number $i = 0$.
- Each round i proceeds as follows:
 - The challenger receives $(\text{id}_1, \text{val}_1, \dots, \text{id}_n, \text{val}_n)$ from $\mathcal{A}$.
 - The challenger sets $w_i = 1$. w_i represents whether all of the writes in this round were valid.
 - For $j \in [n]$, the challenger:
 - * Checks if there exists sk_{id_j} . If not, the challenger samples a new signing key sk_{id_j} .
 - * The adversary plays the role of the server to run:

$w_i \leftarrow w_i \wedge \text{Client.Write}(\text{config}, \text{sk}_{\text{id}_j}, \text{id}_j, \text{val}_j, i)$

- * Sets $\text{dict}[\text{id}_j] \leftarrow \text{val}_j$.
- The challenger gets $(\text{dgst}_i, \pi_i, M)$ from $\mathcal{A}$.
- The challenger runs
 - $b_{i,\text{id}} \leftarrow \text{Client.Monitor}(\text{config}, \text{dgst}_i, \text{id}, \text{dict}[\text{id}], i)$
 - for each $\text{id} \in M$ with $\mathcal{A}$ playing the role of the server. For $\text{id} \in \text{dict}, \text{id} \notin M$, the challenger sets $b_{i,\text{id}} \leftarrow 1$.
- The challenger runs

$\text{ret}_{\text{id}}^{(i)} \leftarrow \text{Client.Read}(\text{config}, \text{dgst}_i, i)$

for each $\text{id} \in \text{dict}$ with $\mathcal{A}$ playing the role of the server.

- Initialize dictionary dict_i with the contents of dict .
- The adversary sends “continue” or “finish”. If the challenger receives “finish”, break out of the loop. Otherwise, $i \leftarrow i + 1$ and continue to the next round.
- The challenger runs
 - $b_{i,\text{id}} \leftarrow \text{Client.Monitor}(\text{config}, \text{dgst}_i, \text{id}, \text{dict}[\text{id}], i)$
 - for each $\text{id} \in \text{dict}$ with $\mathcal{A}$ playing the role of the server.
 - The challenger sets $i \leftarrow i + 1$ and gets (dgst_i, π_i) from $\mathcal{A}$.
 - The challenger runs

$\text{ret}_{\text{id}}^{(i)} \leftarrow \text{Client.Read}(\text{config}, \text{dgst}_i, i)$

for each $\text{id} \in \text{dict}$ with $\mathcal{A}$ playing the role of the server.

Let $R \leftarrow i - 1$ (the number of rounds). The adversary wins if:

- 1) $\text{Auditor.Audit}(\text{config}, \text{dgst}_i, \text{dgst}_{i+1}, \pi_{i+1}) = 1$ and $w_i = 1$ for $i \in [R]$,
- 2) for all $\text{id} \in \text{dict}$ and $i \in [R]$, $b_{i,\text{id}} = 1$, and
- 3) there exists $i \in [R]$, $\text{id} \in \text{dict}$ such that $\text{dict}_i[\text{id}] \neq \text{ret}_{\text{id}}^{(i+1)}$.

Figure 14: Security game for a configurable transparency log.

Experiment 7. The below security game is modified from Experiment 6 so that the adversary only wins if the challenger outputs an old value for a given identifier as a result of a read.

The game proceeds as in Experiment 6 and only the win condition is different.

Let $R \leftarrow i - 1$ (the number of rounds). The adversary wins if:

- 1) $\text{Auditor.Audit}(\text{config}, \text{dgst}_i, \text{dgst}_{i+1}, \pi_{i+1}) = 1$ and $w_i = 1$ for $i \in [R]$,
- 2) for all $\text{id} \in \text{dict}$ and $i \in [R]$, $b_{i,\text{id}} = 1$, and
- 3) there exists $i, j \in [R]$, $\text{id} \in \text{dict}$ such that $\text{dict}_j[\text{id}] = \text{ret}_{\text{id}}^i$ and $i > j$.

Figure 15: Hybrid security game, modified from Experiment 6.

Experiment 8. Let $\mathcal{A}_\Delta$ be the adversary that can win the difference in the games of Experiment 6 and Experiment 7. The game proceeds as in Experiment 6 and only the win condition is different.

Let $R \leftarrow i - 1$ (the number of rounds). The adversary $\mathcal{A}_\Delta$ wins if:

- 1) $\text{Auditor.Audit}(\text{config}, \text{dgst}_i, \text{dgst}_{i+1}, \pi_{i+1}) = 1$ and $w_i = 1$ for $i \in [R]$,
- 2) for all $\text{id} \in \text{dict}$ and $i \in [R]$, $b_{i,\text{id}} = 1$, and
- 3) there exists $i \in [R]$, $\text{id} \in \text{dict}$ such that $\text{dict}_i[\text{id}] \neq \text{ret}_{\text{id}}^{(i+1)}$ and $\forall j < i$, $\text{dict}_j[\text{id}] \neq \text{ret}_{\text{id}}^{(i+1)}$.

Figure 16: Security game representing the difference between Experiment 6 and Experiment 7.

false), and that all monitoring and auditing checks passed. In Client.Read , Client.Write , Client.Monitor , and Auditor.Audit , there are a series of checks performed by the client and the auditor that the adversary must clear before convincing the client to read an old value (stored in the variable ok). This means that the adversary did one of the following:

- (1) Generated a prefix tree proof for the query base tree attesting to (id, val') but the query base tree contains (id, val) :

$\text{Client.Read}(\text{config}, \text{dgst}, \text{id}, t)$

3: ...

4: $\text{val}', \text{Path}_{\text{id}} \leftarrow \text{Server}[\alpha].\text{QueryBT}.\text{GetPath}(\text{id})$

5: $\text{ok} \leftarrow \text{ok} \wedge \text{CheckPath}(\text{root}, \text{id}, \text{vals}, \text{Path}_{\text{id}})$

6: ...

By Lemma 3, the adversary cannot do this without finding a hash collision.

- (2) Generated a prefix tree proof for an update tree attesting to (id, val') , but the update tree did not contain such an update:

$\text{Client.Read}(\text{config}, \text{dgst}, \text{id}, t)$

22: ...

23: $\text{val}_u, \text{Path}_{\text{id},u} \leftarrow \text{tree}_u.\text{GetPath}(\text{id})$

24: $\text{ok} \leftarrow \text{ok} \wedge \text{CheckPath}(\text{root}_u, \text{id}, \text{val}_u, \text{Path}_{\text{id},u})$

25: ...

By Lemma 3, the adversary cannot do this without finding a hash collision.

$\mathcal{B} = (\mathcal{A}_\Delta, \text{Challenger}_{\text{EUF-CMA}})$

- $\text{Challenger}_{\text{EUF-CMA}}$ is the challenger in the EUF-CMA game.
- $\mathcal{B}$ acts as the challenger in the game played with $\mathcal{A}_\Delta$ (Experiment 8).
- $\mathcal{B}$ receives public key pk from $\text{Challenger}_{\text{EUF-CMA}}$.
- $\mathcal{B}$ sets id_{pk} to be empty.
- $\mathcal{B}$ modifies $\text{Sign}(\text{sk}, \text{val})$ to be $\mathcal{B}$ sending val as the chosen message to $\text{Challenger}_{\text{EUF-CMA}}$ and receiving back σ when $\text{sk} = \text{pk}$. For all other values of sk , $\mathcal{B}$ runs Sign unmodified to produce a signature (since it knows the signing key).
- Each round i proceeds as follows:
 - $\mathcal{B}$ receives $(\text{id}_1, \text{val}_1, \dots, \text{id}_n, \text{val}_n)$ from $\mathcal{A}_\Delta$.
 - For each $j \in [n]$, $\mathcal{B}$:
 - * If id_{pk} is empty, then set id_{pk} to be id_j .
 - * If $\text{id}_j = \text{id}_{\text{pk}}$, $\mathcal{B}$ then runs (with $\mathcal{A}_\Delta$ playing the role of the server)

$$w_i \leftarrow w_i \wedge \text{Client.Write}(\text{config}, \text{pk}, \text{id}_j, \text{val}_j, i)$$
 - * Otherwise, $\mathcal{B}$ checks if there exists sk_{id_j} . If not, the $\mathcal{B}$ samples a new signing key sk_{id_j} . Then the adversary plays the role of the server to run:

$$w_i \leftarrow w_i \wedge \text{Client.Write}(\text{config}, \text{sk}_{\text{id}_j}, \text{id}_j, \text{val}_j, i)$$
 - * Sets $\text{dict}[\text{id}_j] \leftarrow \text{val}_j$.
 - The rest of the round proceeds as defined in Experiment 6.
- The rest of the game proceeds as defined in Experiment 6.
- If the adversary $\mathcal{A}_\Delta$ wins, there exists $i \in [R]$, $\text{id} \in \text{dict}$ such that $\text{dict}_i[\text{id}] \neq \text{ret}_{\text{id}}^{(i+1)}$ and $\forall j < i, \text{dict}_j[\text{id}] \neq \text{ret}_{\text{id}}^{(i+1)}$. If $\text{id} = \text{id}_{\text{pk}}$, $\mathcal{B}$ simply sends $\text{ret}_{\text{id}}^{(i+1)}$ and its associated signature to $\text{Challenger}_{\text{EUF-CMA}}$.

With an $\mathcal{A}_\Delta$ that can win Experiment 8, wins the EUF-CMA game with advantage $1/\text{poly}(\cdot)$ because the invalid read that $\mathcal{A}_\Delta$ forges a signature and one of the ids uses the signatures from $\text{Challenger}_{\text{EUF-CMA}}$.

Figure 17: Adversary $\mathcal{B}$ that can be constructed from $\mathcal{A}_\Delta$ to win the EUF-CMA game.

- (3) Generated a chronological tree proof attesting that the most recent addition to the update log is a prefix tree containing (id, val') :

$\text{Client.Read}(\text{config}, \text{dgst}, \text{id}, t)$

19: ...

20: $(\text{tree}_u, \text{ok}_u) \leftarrow \text{dgst}[\alpha].\text{UpdateLog}[u]^*$

21: $\text{ok} \leftarrow \text{ok} \wedge \text{ok}_u$

22: ...

By Lemma 4, the adversary cannot do this without finding a hash collision.

- (4) Generated an extension proof attesting that the update log containing a prefix tree containing (id, val') is an extension of an update log containing a prefix tree containing (id, val) :

$\text{Client.Write}(\text{config}, \text{sk}, \text{id}, \text{val}, t) \rightarrow \{0, 1\}$

2: ...

3: $\text{Server}[\alpha].\text{Write}(\text{id}, (\text{val}, \sigma))$

4: Wait until epoch $t + 1$

5: $\text{dgst} \leftarrow \text{Auditor.GetDigest}(t + 1)$

6: $\text{val}_{\text{latest}} \leftarrow \text{Client.Read}(\text{config}, \text{dgst}, \text{id}, t + 1)$

7: ...

$\text{Auditor.Audit}(\text{config}, \text{dgst}_{\text{old}}, \text{dgst}_{\text{new}}, \pi) \rightarrow \{0, 1\}$:

11: ...

12: **if** $\text{dgst}_{\text{old}}[\alpha].t \bmod \text{config}.t_v =$

$\text{dgst}_{\text{new}}[\alpha].t \bmod \text{config}.t_v$ **then**

13: $\text{ok} \leftarrow \text{ok} \wedge \text{VerifyExtension}(\pi[\alpha].ul,$

$\text{dgst}_{\text{old}}[\alpha].ulroot, \text{dgst}_{\text{new}}[\alpha].ulroot)$

14: **end if**

15: ...

By Lemma 5, the adversary cannot do this without finding a hash collision.

- (5) Created a collision in the hash chain of base prefix trees (so the client did not detect the equivocation in a later epoch):

$\text{Auditor.Audit}(\text{config}, \text{dgst}_{\text{old}}, \text{dgst}_{\text{new}}, \pi) \rightarrow \{0, 1\}$:

5: ...

6: $h' \leftarrow \text{dgst}_{\text{old}}[\alpha].\text{HashChain}$

7: **for** $n \in \pi[\alpha].hc$ **do**

8: $h' \leftarrow H(n, h')$

9: **end for**

10: $\text{ok} \leftarrow \text{ok} \wedge h' \stackrel{?}{=} \text{dgst}_{\text{new}}[\alpha].\text{HashChain}$

11: ...

Thus, if the adversary does win, then we can construct an adversary $\mathcal{C}$ that can find a collision in the hash function.

Case 2. We now argue security for when $\text{agghistory} = \text{true}$. Assume that the challenger reads an old value val' for id when the last write to id was val ($\text{agghistory} = \text{true}$), and that all monitoring and auditing checks passed. This means that the adversary did one of the following:

- * (1) through (4) are the same as in Case 1.

- (5) Each read and monitor invocation query a number of prefix trees that have been generated since the most recent write, and for a given identifier id , if the monitor call happens after the read call, then the set of prefix trees queried must intersect.

$\text{Client.Read}(\text{config}, \text{dgst}, \text{id}, t)$

6: ...

7: **for** $\text{root}_h \in \text{Server}[\alpha].\text{HistoryForest.roots}$ **do**

8: $t_h \leftarrow$ verification period root_h corresponds to

9: $\text{val}_h, \text{Path}_{\text{id}, h} \leftarrow \text{Server}[\alpha].\text{HistoryForest}[t_h].\text{GetPath}(\text{id})$

10: $\text{ok} \leftarrow \text{ok} \wedge \text{CheckPath}(\text{root}_h, \text{id}, \text{val}_h, \text{Path}_{\text{id}, h})$

11: $\text{ok} \leftarrow \text{ok} \wedge \text{val}_h.\text{vals} \stackrel{?}{\subseteq} \text{val}.\text{vals}$

12: **end for**

13: ...

$\text{Client.Monitor}(\text{config}, \text{dgst}, \text{id}, \text{val}, t) \rightarrow \{0, 1\}$

7: ...

8: $t' \leftarrow$ verification period id was updated to val

9: **for** $t_l \in$ path from t' to root in $\text{Server}[\alpha].\text{HistoryForest}$ **do**

10: $\text{dgst}_{t_l} \leftarrow \text{Auditor.GetDigest}(t_l)$

11: $\text{val}', \text{Path} \leftarrow \text{Server}[\alpha].\text{HistoryForest}[t_l].\text{GetPath}(\text{id})$

12: $\text{ok} \leftarrow \text{ok} \wedge \text{CheckPath}(\text{dgst}_{t_l}[\alpha].vroot, \text{id}, \text{val}', \text{Path})$

13: $\text{ok} \leftarrow \text{ok} \wedge \text{val}' \stackrel{?}{=} \text{val}$

14: **end for**

15: ...

Because the prefix trees are stored in a chronological forest, if the adversary can convince the challenger that this set of prefix trees does not overlap, then it has found a collision in the hash function.

Given an overlapping set of prefix trees, the adversary then must produce a prefix tree proof attesting to (id, val') when the last write to id was for $val \neq val'$.

Based on the security of prefix trees (Lemma 3), an adversary that can do this has found a collision in the hash function.

If the adversary does win in this case (when `agghistory = true`), then we can construct an adversary $\mathcal{C}$ that can find a collision in the hash function. This completes the proof.

□